

MERN Stack Interview Master - Numbered Detailed Q&A;

[HTML](#) | [CSS](#) | [JavaScript](#) | [React](#) | [Node.js](#) | [Express.js](#) | [MongoDB](#) | [Backend](#) | [MERN](#) | [Projects](#) | [Coding](#) | [Advanced Topics](#) | [HR](#)

Interview method: Read the answer aloud. Then close the PDF and explain the same concept in your own words. For every code example, know what each line does and why it is used.

HTML

1. What is HTML?

Easy answer: HTML (HyperText Markup Language) is the markup language used to structure a web page. It tells the browser what is a heading, paragraph, image, form, link, table, and so on. HTML gives structure; CSS gives appearance; JavaScript gives behavior.

Code / example:

```
<h1>My Portfolio</h1>
<p>I am a MERN developer.</p>
```

2. What is HTML5?

Easy answer: HTML5 is the modern HTML standard. It added semantic elements, native audio/video, improved form controls and browser APIs, making web development more expressive and practical.

Code / example:

```
<video controls src="intro.mp4"></video>
```

3. What is an HTML tag?

Easy answer: A tag is markup such as `<p>` or `</p>`. An element is usually the complete structure, such as `<p>Hello</p>`.

Code / example:

```
<p>Hello</p>
```

4. What is an HTML element?

Easy answer: An HTML element is a complete unit of markup. It may contain an opening tag, content, and a closing tag; some elements are void elements and do not have closing tags.

Code / example:

```

```

5. What is an attribute?

Easy answer: An attribute gives additional information or behavior to an HTML element. Common attributes include id, class, href, src, alt, name, value, disabled and required.

Code / example:

```
<a href="/about" class="link">About</a>
```

6. What is `<!DOCTYPE html>`?

Easy answer: It tells the browser to parse the document using modern standards mode rather than old quirks behavior.

Code / example:

```
<!DOCTYPE html>
<html> ... </html>
```

7. What are block-level elements?

Easy answer: They normally start on a new line and can take the available width. Examples include div, p, h1, section, article and main.

Code / example:

```
<div>One block</div>
<div>Another block</div>
```

8. What are inline elements?

Easy answer: They normally remain in the same line and take only the space they need. Examples include span, a, strong and em.

Code / example:

```
<p>Hello <span>world</span></p>
```

9. div vs span?

Easy answer: div is a generic block container; span is a generic inline container. Both are non-semantic and should not replace semantic elements when a meaningful element exists.

Code / example:

```
<div class="card">Hello <span class="highlight">world</span></div>
```

10. p vs div?

Easy answer: p represents a paragraph and has semantic meaning. div is a generic container with no paragraph meaning. Use p for actual paragraphs and div for grouping/layout.

Code / example:

```
<p>This is a paragraph.</p>
<div class="row">...</div>
```

11. What are void/empty elements?

Easy answer: Void elements do not have closing tags because they cannot contain child content. Examples include img, input, br, hr, meta and link.

Code / example:

```
<input type="email">

```

12. What is nesting?

Easy answer: Nesting means putting elements inside other elements. Proper nesting keeps the document tree valid and predictable.

Code / example:

```
<div>
  <p>Hello</p>
</div>
```

13. What are global attributes?

Easy answer: Attributes that can be used on most HTML elements include id, class, title, hidden, style, lang, tabindex and data-* attributes.

Code / example:

```
<div id="app" class="dark" title="Main app">...</div>
```

14. What is id?

Easy answer: id gives an element a unique identifier within the document. It is useful for CSS, JavaScript and labels/anchors.

Code / example:

```
<input id="email">
<label for="email">Email</label>
```

15. What is class?

Easy answer: class groups elements so the same CSS or JavaScript behavior can apply to multiple elements. An element can have multiple classes.

Code / example:

```
<button class="btn primary large">Save</button>
```

16. Can multiple elements share one id?

Easy answer: No. IDs are intended to be unique within a document. Reusing an id can break CSS targeting, accessibility relationships and JavaScript selectors.

Code / example:

```
<!-- Prefer class for repeated elements -->
<button class="btn">One</button>
<button class="btn">Two</button>
```

17. Can an element have multiple classes?

Easy answer: Yes. Separate class names with spaces. This is commonly used to combine reusable utility and component styles.

Code / example:

```
<div class="card shadow rounded">...</div>
```

18. What is semantic HTML?

Easy answer: Semantic HTML uses elements whose names describe their meaning, such as header, nav, main, article, section, aside and footer. It improves readability, accessibility and machine understanding.

Code / example:

```
<header>Logo</header>
<nav>Links</nav>
<main>Content</main>
<footer>Copyright</footer>
```

19. Why is semantic HTML important?

Easy answer: It gives structure to humans, browsers, screen readers and search engines. A button is better than a clickable div because browsers already know how a button behaves and how assistive technologies should announce it.

Code / example:

```
<button type="button">Submit</button>
```

20. What is accessibility?

Easy answer: Accessibility means making a website usable for people with different abilities. Use semantic HTML, labels, keyboard support, useful focus states, alt text and appropriate ARIA only when needed.

Code / example:

```
<label for="name">Name</label>
<input id="name" name="name">
```

21. What is ARIA?

Easy answer: ARIA (Accessible Rich Internet Applications) adds accessibility semantics when native HTML is not enough. Prefer native elements first; do not use ARIA to imitate behavior that a real HTML element already provides.

Code / example:

```
<button aria-label="Close dialog">X</button>
```

22. Why use alt on images?

Easy answer: alt gives a text alternative for images. Screen readers can announce it and it can be shown when an image cannot load. Decorative images can use alt="".

Code / example:

```

```

23. What is an HTML form?

Easy answer: A form groups controls used to collect input. Inputs should have useful names and labels so the browser and server can understand the submitted values.

Code / example:

```
<form action="/login" method="post">
  <input name="email" type="email">
  <button type="submit">Login</button>
</form>
```

24. What is the name attribute in forms?

Easy answer: name is the key used for a successful form control when data is submitted. JavaScript-controlled forms can still use name to keep data models clear.

Code / example:

```
<input name="email" type="email">
```

25. GET vs POST in an HTML form?

Easy answer: GET places form data in the URL query string and is commonly used for retrieval/search. POST puts data in the request body and is commonly used for submitting or creating data. Neither method by itself provides security; HTTPS and proper backend validation are still required.

Code / example:

```
<form method="get" action="/search">...</form>
<form method="post" action="/users">...</form>
```

26. What are common input types?

Easy answer: Common types include text, email, password, number, date, checkbox, radio, file, url and submit. Choosing the correct type improves built-in validation and mobile keyboard behavior.

Code / example:

```
<input type="email">
<input type="date">
<input type="checkbox">
```

27. What is client-side form validation?

Easy answer: It checks data in the browser before submission. It improves user experience, but it is not a security boundary because users can bypass browser checks; always validate on the server too.

Code / example:

```
<input type="email" required>
<input type="text" minlength="3">
```

28. What are required, min, max and pattern?

Easy answer: They are native validation attributes. required makes a field mandatory; min/max constrain numbers or other applicable values; pattern matches text against a regular expression.

Code / example:

```
<input type="text" pattern="[A-Za-z]+" required>
<input type="number" min="1" max="100">
```

29. Radio vs checkbox?

Easy answer: Radio buttons are usually used when one choice should be selected from a group sharing the same name. Checkboxes allow independent yes/no or multiple selections.

Code / example:

```
<input type="radio" name="role" value="candidate">
<input type="radio" name="role" value="employer">
<input type="checkbox" name="skills" value="react">
```

30. What is select?

Easy answer: select creates a dropdown list. option elements provide choices.

Code / example:

```
<select name="role">
  <option value="candidate">Candidate</option>
  <option value="employer">Employer</option>
</select>
```

31. What is textarea?

Easy answer: textarea is used for multi-line text such as descriptions, comments and messages.

Code / example:

```
<textarea name="description" rows="5"></textarea>
```

32. submit button vs normal button?

Easy answer: A submit button submits its associated form. A button with type="button" does not submit the form and is usually used for JavaScript actions.

Code / example:

```
<button type="submit">Save</button>
<button type="button">Open modal</button>
```

33. How do you open a link in a new tab?

Easy answer: Use target="_blank". For an untrusted external destination, adding rel="noopener noreferrer" is a useful defense-in-depth practice.

Code / example:

```
<a href="https://example.com" target="_blank" rel="noopener noreferrer">Open</a>
```

34. How do you add audio and video?

Easy answer: HTML provides native audio and video elements. Controls let the browser show playback controls.

Code / example:

```
<audio controls src="song.mp3"></audio>
<video controls src="movie.mp4"></video>
```

35. strong vs b?

Easy answer: strong gives semantic importance. b primarily draws attention visually without carrying the same importance meaning. Use the element whose meaning matches the content.

Code / example:

```
<strong>Warning</strong>
<b>Keyword</b>
```

36. em vs i?

Easy answer: em represents emphasis. i can represent text in an alternate voice, such as a technical term or idiomatic phrase, depending on context.

Code / example:

```
<em>Very important</em>
<i>Term</i>
```

37. section vs article?

Easy answer: section groups related content around a theme, often with a heading. article represents a self-contained piece that could stand on its own, such as a post or news item.

Code / example:

```
<section><h2>Projects</h2>...</section>
<article><h2>Post</h2>...</article>
```

38. What is SEO and how does HTML help it?

Easy answer: SEO is Search Engine Optimization. Meaningful title/headings, semantic elements, descriptive links, useful content, alt text and correct metadata help search engines understand the page.

Code / example:

```
<title>MERN Developer Portfolio</title>
<meta name="description" content="Portfolio of a MERN developer">
```

39. What are meta tags?

Easy answer: Meta elements provide metadata about the document, such as character encoding, viewport settings and description. They live in head and are not usually visible as page content.

Code / example:

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

CSS

40. What is CSS?

Easy answer: CSS controls presentation and layout: colors, spacing, typography, responsive behavior, positioning, transitions and animations.

Code / example:

```
body { font-family: Arial; }
.card { padding: 20px; }
```

41. What are the ways to apply CSS?

Easy answer: Inline styles are on an element, internal CSS is in a style tag, and external CSS is in a separate file. External CSS is usually best for maintainability and reuse.

Code / example:

```
<link rel="stylesheet" href="style.css">
```

42. What is a CSS selector?

Easy answer: A selector chooses which elements receive a CSS rule. Selectors include elements, classes, IDs, attributes, pseudo-classes and combinators.

Code / example:

```
.card { padding: 20px; }
button:hover { opacity: .8; }
```

43. What is the universal selector?

Easy answer: The * selector matches all elements. It is commonly used for a simple reset such as box-sizing.

Code / example:

```
* { box-sizing: border-box; }
```

44. What are class and ID selectors?

Easy answer: .class targets elements with a class; #id targets one element with that ID. Classes are normally better for reusable styling.

Code / example:

```
.card { }
#header { }
```

45. What are CSS combinators?

Easy answer: Combinators describe relationships between elements: descendant (space), child (>), adjacent sibling (+), and general sibling (~).

Code / example:

```
nav a { color: blue; }
.list > li { margin: 4px; }
```

46. What is specificity?

Easy answer: Specificity is the selector priority used when declarations compete. A simplified order is inline styles > IDs > classes/attributes/pseudo-classes > elements/pseudo-elements, with !important as a special case.

Code / example:

```
#title { color: red; }
```

```
.title { color: blue; }
```

47. What is !important?

Easy answer: It marks a declaration as important and changes the normal cascade. Overusing it makes styles harder to override, so prefer fixing specificity or structure instead.

Code / example:

```
.error { color: red !important; }
```

48. What is inheritance in CSS?

Easy answer: Some properties naturally inherit from a parent, such as color and font-family. Layout properties like width and margin generally do not inherit automatically.

Code / example:

```
body { font-family: Arial; color: #222; }  
/* children inherit these */
```

49. What is the cascade?

Easy answer: The cascade is the browser process that chooses the winning CSS declaration based on origin/importance, specificity and source order. This is why two matching rules do not both win equally.

Code / example:

```
.btn { color: black; }  
.btn { color: blue; } /* wins when specificity is same and this comes later */
```

50. Explain the box model.

Easy answer: The box model consists of content, padding, border and margin. A useful mental model is: total size = content + padding + border + margin (depending on box-sizing).

Code / example:

```
.box { width: 200px; padding: 20px; border: 2px solid; margin: 10px; }
```

51. Margin vs padding?

Easy answer: Padding is inside the border and increases inner breathing room. Margin is outside the border and creates distance from other elements.

Code / example:

```
.card { padding: 20px; margin: 20px; }
```

52. What is box-sizing border-box?

Easy answer: With border-box, the declared width and height include padding and border. This makes responsive sizing easier because the outer box does not unexpectedly grow.

Code / example:

```
*, *::before, *::after { box-sizing: border-box; }
```

53. display:block vs inline vs inline-block?

Easy answer: block starts a new line and generally accepts width/height; inline stays in text flow and ignores some box dimensions; inline-block stays inline but accepts width/height.

Code / example:

```
.a { display: block; }  
.b { display: inline; }  
.c { display: inline-block; }
```

54. display:none vs visibility:hidden?

Easy answer: display:none removes the element from layout. visibility:hidden hides the element but normally keeps its layout space.

Code / example:

```
.hidden1 { display: none; }  
.hidden2 { visibility: hidden; }
```

55. What is Flexbox?

Easy answer: Flexbox is a one-dimensional layout model for arranging items along a main axis and cross axis. It is excellent for alignment, navigation bars, cards and small component layouts.

Code / example:

```
.row { display: flex; gap: 16px; align-items: center; }
```

56. What is the main axis and cross axis?

Easy answer: The main axis follows flex-direction. With row, main axis is horizontal and cross axis is vertical. With column, the axes swap.

Code / example:

```
.box { display:flex; flex-direction: column; }
```

57. justify-content vs align-items?

Easy answer: justify-content distributes items along the main axis. align-items aligns items along the cross axis. Their visual directions change when flex-direction changes.

Code / example:

```
.box { display:flex; justify-content:center; align-items:center; }
```

58. What is flex-wrap?

Easy answer: It controls whether flex items stay on one line or wrap onto multiple lines.

Code / example:

```
.cards { display:flex; flex-wrap: wrap; gap: 12px; }
```

59. What is CSS Grid?

Easy answer: Grid is a two-dimensional layout system for rows and columns. It is often better than Flexbox for page layouts or card grids where both dimensions matter.

Code / example:

```
.grid { display:grid; grid-template-columns: repeat(3, 1fr); gap:20px; }
```

60. Flexbox vs Grid?

Easy answer: Use Flexbox when the layout is mainly one-dimensional. Use Grid when you need row and column control at the same time. They can also be combined.

Code / example:

```
.layout { display:grid; }  
.nav { display:flex; }
```

61. Explain position values.

Easy answer: static is normal flow. relative keeps its space and can be offset. absolute is positioned relative to a suitable containing block. fixed is attached to the viewport. sticky behaves like normal flow until a scroll threshold is reached.

Code / example:

```
.parent { position: relative; }  
.child { position: absolute; top: 0; right: 0; }
```

62. What is z-index?

Easy answer: z-index controls stacking order within the relevant stacking context. It does not make an element magically jump above every ancestor or stacking context.

Code / example:

```
.modal { position: fixed; z-index: 1000; }
```

63. What is responsive design?

Easy answer: Responsive design means the layout adapts to different screen sizes. Use flexible widths, Flexbox/Grid, responsive images and media queries rather than hard-coding one desktop size.

Code / example:

```
@media (max-width: 768px) { .grid { grid-template-columns: 1fr; } }
```

64. What are px, %, em, rem, vh and vw?

Easy answer: px is a CSS pixel unit; % is relative to a relevant containing value; em is relative to a font-size context; rem is relative to the root font size; vh/vw relate to viewport height/width.

Code / example:

```
html { font-size: 16px; }  
.title { font-size: 2rem; }  
.hero { min-height: 100vh; }
```

65. What are pseudo-classes?

Easy answer: Pseudo-classes describe states or conditions, such as :hover, :focus, :checked, :first-child and :disabled.

Code / example:

```
button:hover { transform: scale(1.03); }  
input:focus { outline: 2px solid blue; }
```

66. What are pseudo-elements?

Easy answer: Pseudo-elements represent a conceptual part of an element, such as ::before, ::after, ::first-letter and ::placeholder.

Code / example:

```
.card::before { content:""; display:block; }
```

67. Pseudo-class vs pseudo-element?

Easy answer: A pseudo-class usually describes a state or condition; a pseudo-element targets a generated or conceptual part of an element.

Code / example:

```
a:hover { color:red; }  
a::after { content:"■"; }
```

68. What is a CSS transition?

Easy answer: A transition smoothly interpolates a CSS property when its value changes. It is ideal for hover/focus interactions.

Code / example:

```
button { transition: transform .2s ease; }  
button:hover { transform: translateY(-2px); }
```

69. What is CSS animation?

Easy answer: Animations use keyframes to define multiple stages and can run automatically, repeat and have timing controls.

Code / example:

```
@keyframes spin { to { transform: rotate(360deg); } }  
.loader { animation: spin 1s linear infinite; }
```

70. Transition vs animation?

Easy answer: A transition is normally triggered by a change between two states. An animation can define many keyframes and run without a direct state change.

Code / example:

```
/* transition */  
.btn { transition: all .2s; }  
/* animation */  
@keyframes fade { from{opacity:0} to{opacity:1} }
```

71. What is transform?

Easy answer: transform changes an element's visual geometry without changing normal document flow. Common functions are translate, scale, rotate and skew.

Code / example:

```
.card:hover { transform: translateY(-5px) scale(1.01); }
```

72. What is overflow?

Easy answer: overflow controls what happens when content exceeds the box. Common values are visible, hidden, auto and scroll.

Code / example:

```
.panel { max-height: 300px; overflow: auto; }
```

73. What are CSS variables?

Easy answer: Custom properties store reusable values. They help with themes and consistent spacing/colors.

Code / example:

```
:root { --primary: #2563eb; --gap: 16px; }  
.btn { background: var(--primary); gap: var(--gap); }
```

74. opacity:0 vs display:none?

Easy answer: opacity:0 makes an element fully transparent but it can still occupy layout space and may still receive interaction depending on the rest of the CSS. display:none removes it from layout and rendering participation.

Code / example:

```
.fade { opacity: 0; }  
.removed { display:none; }
```

JavaScript - Core & Intermediate

75. What is JavaScript?

Easy answer: JavaScript is a programming language used for application logic. In a MERN stack, it can run in the browser (React) and on the server (Node.js).

Code / example:

```
const message = "Hello";
console.log(message);
```

76. Is JavaScript interpreted or compiled?

Easy answer: Modern JavaScript engines use a mix of parsing, interpretation and just-in-time compilation/optimization. For an interview, say: JavaScript is executed by an engine such as V8, which optimizes code at runtime.

Code / example:

```
const total = 10 + 20;
```

77. Is JavaScript synchronous or asynchronous?

Easy answer: JavaScript execution is synchronous by default, meaning statements run one after another. The runtime provides asynchronous APIs such as timers, network requests and Promises so programs can handle I/O without blocking the main flow.

Code / example:

```
console.log("A");
setTimeout(()=>console.log("B"), 0);
console.log("C");
```

78. What are primitive data types?

Easy answer: String, Number, BigInt, Boolean, Undefined, Null and Symbol are primitive types. Primitives are immutable values.

Code / example:

```
let name = "Abhay";
let age = 22;
let ok = true;
```

79. What are reference types?

Easy answer: Objects, arrays and functions are reference-type values. Variables store references to these objects, so two variables can point at the same object.

Code / example:

```
const a = {x:1};
const b = a;
b.x = 2;
console.log(a.x); // 2
```

80. What is dynamic typing?

Easy answer: JavaScript variables do not have fixed declared types. A variable can hold different value types at different times, although good code keeps types predictable.

Code / example:

```
let value = 10;
value = "ten";
```

81. What is type coercion?

Easy answer: Type coercion is converting a value from one type to another, sometimes implicitly. == performs coercion more often; === does not.

Code / example:

```
"5" + 1; // "51"
"5" - 1; // 4
```

82. null vs undefined?

Easy answer: undefined generally means a value is missing/unassigned. null is an intentional empty value chosen by the programmer.

Code / example:

```
let a; // undefined
let b = null; // intentional empty
```

83. What is NaN?

Easy answer: NaN means Not-a-Number. It is a special Number value that represents an invalid numeric result.

Code / example:

```
const x = Number("abc");
```

```
console.log(Number.isNaN(x)); // true
```

84. What is var?

Easy answer: var is function-scoped, not block-scoped, and allows redeclaration. Its declaration is hoisted and initialized as undefined before execution reaches the assignment.

Code / example:

```
function test(){
  if(true){ var x=10; }
  console.log(x); // 10
}
```

85. What is let?

Easy answer: let is block-scoped and can be reassigned. Accessing it before initialization causes a ReferenceError because of the temporal dead zone.

Code / example:

```
let count = 0;
count = 1;
```

86. What is const?

Easy answer: const creates a block-scoped binding that cannot be reassigned. The object referenced by a const variable can still be mutated unless you freeze it or use immutable update patterns.

Code / example:

```
const user={name:"A"};
user.name="B"; // allowed
```

87. What is the difference between == and ===?

Easy answer: == allows type coercion before comparison. === compares without implicit type conversion, so both type and value must match. Prefer === in normal application code.

Code / example:

```
5 == "5"; // true
5 === "5"; // false
```

88. What is scope?

Easy answer: Scope determines where a variable can be accessed. Important scopes are global, function and block scope.

Code / example:

```
let globalValue = 1;
function f(){ const localValue=2; }
```

89. What is lexical scope?

Easy answer: Lexical scope means the accessibility of variables is determined by where functions and blocks are written in source code, not by where they are called.

Code / example:

```
const x=10;
function show(){ console.log(x); }
```

90. What is hoisting?

Easy answer: JavaScript processes declarations before running code in that scope. var declarations become available as undefined, while let/const are inaccessible before initialization.

Code / example:

```
console.log(a); // undefined
var a=10;
```

91. What is the temporal dead zone?

Easy answer: The temporal dead zone is the time from entering a scope until a let/const variable is initialized. Access during that time throws a ReferenceError.

Code / example:

```
console.log(x); // ReferenceError
let x=10;
```

92. Function declaration vs function expression?

Easy answer: A declaration creates a named function statement and is hoisted with its function value. A function expression is assigned to a variable and follows the variable's initialization rules.

Code / example:

```
sayHi();
```

```
function sayHi(){ console.log("hi"); }

const sayBye = function(){ console.log("bye"); };
```

93. What is an arrow function?

Easy answer: Arrow functions provide shorter function syntax and capture this from the surrounding lexical scope. They are not suitable when you need a function-specific this, arguments or constructor behavior.

Code / example:

```
const add = (a,b) => a+b;
```

94. Arrow function vs normal function?

Easy answer: Arrow functions do not have their own this, arguments object or prototype and cannot be used with new. Normal functions have dynamic this behavior based on the call site.

Code / example:

```
const obj={
  x:10,
  normal(){ return this.x; },
  arrow:()=>this.x
};
```

95. What is a callback?

Easy answer: A callback is a function passed to another function to be called later or as part of an operation. Callbacks are fundamental to array methods and asynchronous APIs.

Code / example:

```
[1,2,3].map(x => x*2);
```

96. What is a higher-order function?

Easy answer: A higher-order function accepts a function, returns a function, or both. map, filter, reduce and setTimeout are common examples of APIs that work with functions.

Code / example:

```
function apply(fn, value){ return fn(value); }
apply(x=>x*2, 5);
```

97. What is a pure function?

Easy answer: A pure function produces the same output for the same input and has no observable side effects. Pure functions are easier to test and reason about.

Code / example:

```
function add(a,b){ return a+b; }
```

98. What is an IIFE?

Easy answer: An Immediately Invoked Function Expression is a function expression that runs immediately. It was used to create isolated scope before modules and let/const became common.

Code / example:

```
(() => {
  const secret = 42;
  console.log(secret);
})();
```

99. What is a closure?

Easy answer: A closure is a function together with access to variables from its outer lexical scope. The inner function can still use those variables after the outer function has returned.

Code / example:

```
function outer(){
  let count=0;
  return ()=>++count;
}
const counter=outer();
console.log(counter()); // 1
console.log(counter()); // 2
```

100. What is this in JavaScript?

Easy answer: For normal functions, this is determined by how the function is called. In an object method it commonly refers to the object; with call/apply/bind it can be set explicitly. Arrow functions inherit this from their surrounding scope.

Code / example:

```
const user={name:"Abhay", greet(){ console.log(this.name); }};
user.greet();
```

101. What are call, apply and bind?

Easy answer: call invokes immediately with arguments listed separately. apply invokes immediately with arguments supplied as an array-like object. bind returns a new function with a chosen this and optional preset arguments.

Code / example:

```
function greet(age){ console.log(this.name, age); }
const user={name:"Abhay"};
greet.call(user, 22);
greet.apply(user, [22]);
const fn=greet.bind(user, 22);
fn();
```

102. What is destructuring?

Easy answer: Destructuring extracts values from arrays or properties from objects into variables.

Code / example:

```
const user={name:"Abhay", age:22};
const {name, age}=user;
```

103. What is the spread operator?

Easy answer: Spread expands iterable values or object properties. It is useful for copying/combining arrays and creating new objects without mutating the original top-level structure.

Code / example:

```
const a=[1, 2];
const b=[...a, 3];
const user2={...user, age:23};
```

104. What is the rest operator?

Easy answer: Rest collects remaining values into an array or remaining object properties into an object. In function parameters it gathers extra arguments.

Code / example:

```
function sum(...nums){ return nums.reduce((a,b)=>a+b,0); }
```

105. What are default parameters?

Easy answer: They provide a fallback value when an argument is undefined.

Code / example:

```
function greet(name="Guest"){ return `Hi ${name}`; }
```

106. What are template literals?

Easy answer: Template literals use backticks and allow string interpolation with `\${...}`. They also support multi-line strings.

Code / example:

```
const name="Abhay";
console.log(`Hello ${name}`);
```

107. What is optional chaining?

Easy answer: Optional chaining ?. safely accesses nested properties and returns undefined if an earlier part is nullish, instead of throwing.

Code / example:

```
const city = user?.address?.city;
```

108. What is nullish coalescing?

Easy answer: The ?? operator uses the right-hand value only when the left side is null or undefined. Unlike ||, it does not replace valid falsy values such as 0 or an empty string.

Code / example:

```
const page = inputPage ?? 1;
```

109. What is a shallow copy?

Easy answer: A shallow copy creates a new outer array/object but nested objects still share references.

Code / example:

```
const copy = {...user};
const arrCopy=[...arr];
```

110. What is a deep copy?

Easy answer: A deep copy duplicates nested data so changes in the copy do not affect the original. structuredClone is a modern option for supported data types.

Code / example:

```
const clone = structuredClone(original);
```

111. What is Object.freeze?

Easy answer: Object.freeze prevents adding, deleting and changing properties at the first object level. It is shallow; nested objects can still be mutable.

Code / example:

```
const config=Object.freeze({port:5000});
```

112. What is a Promise?

Easy answer: A Promise represents the eventual result of an asynchronous operation. Its main states are pending, fulfilled and rejected.

Code / example:

```
const p = new Promise((resolve,reject)=>{ resolve("done"); });
p.then(console.log);
```

113. What are promise states?

Easy answer: A Promise starts pending, then becomes fulfilled if it succeeds or rejected if it fails. Once settled, it does not change to another state.

Code / example:

```
Promise.resolve("ok");
Promise.reject(new Error("fail"));
```

114. How do then, catch and finally work?

Easy answer: then handles fulfillment, catch handles rejection, and finally runs after settlement regardless of success/failure.

Code / example:

```
fetch("/api/users")
  .then(r=>r.json())
  .then(data=>console.log(data))
  .catch(err=>console.error(err))
  .finally(()=>console.log("done"));
```

115. What is async/await?

Easy answer: async/await is syntax built on Promises. An async function returns a Promise, and await lets the function pause until a Promise settles, making asynchronous code easier to read.

Code / example:

```
async function load(){
  const res=await fetch("/api/users");
  return res.json();
}
```

116. How do you handle errors with async/await?

Easy answer: Use try/catch around awaited operations. Errors thrown by a rejected Promise can be caught there.

Code / example:

```
async function load(){
  try {
    const res=await fetch("/api/users");
    if(!res.ok) throw new Error("Request failed");
    return await res.json();
  } catch(err) {
    console.error(err);
  }
}
```

117. What is Promise.all?

Easy answer: Promise.all runs independent Promises together and fulfills when every Promise fulfills. If one rejects, the returned Promise rejects.

Code / example:

```
const [users,posts]=await Promise.all([
  getUsers(), getPosts()
]);
```

118. Promise.all vs allSettled?

Easy answer: all fails fast when one Promise rejects. allSettled waits for every Promise and returns the status/result of each. Use allSettled when you need every outcome.

Code / example:

```
const results = await Promise.allSettled([job1(), job2(), job3()]);
```

119. Promise.race vs Promise.any?

Easy answer: race settles as soon as the first input Promise settles, whether fulfilled or rejected. any fulfills when the first Promise fulfills and rejects only if every Promise rejects.

Code / example:

```
const fastest = await Promise.race([a(), b()]);
const firstSuccess = await Promise.any([a(), b()]);
```

120. What is the event loop?

Easy answer: JavaScript runs synchronous code on the call stack. When asynchronous work completes, the runtime schedules its continuation in queues. The event loop repeatedly moves ready work to the stack when it is free.

Code / example:

```
console.log("A");
setTimeout(()=>console.log("B"), 0);
console.log("C");
// A C B
```

121. What is the call stack?

Easy answer: The call stack tracks currently executing function calls. A function is pushed when called and popped when it returns. Long synchronous work blocks other JavaScript from running on that thread.

Code / example:

```
function a(){ b(); }
function b(){ console.log("inside"); }
a();
```

122. What is the microtask queue?

Easy answer: Promise reactions and queueMicrotask callbacks use the microtask queue. Microtasks are processed after the current synchronous code and before the next regular task such as a timer callback.

Code / example:

```
Promise.resolve().then(()=>console.log("microtask"));
setTimeout(()=>console.log("task"), 0);
```

123. Why does Promise usually run before setTimeout(0)?

Easy answer: A timer callback is a task. A fulfilled Promise continuation is a microtask. After synchronous code finishes, the runtime drains microtasks before moving to the next task.

Code / example:

```
setTimeout(()=>console.log("B"), 0);
Promise.resolve().then(()=>console.log("C"));
console.log("A");
// A C B
```

124. What is the DOM?

Easy answer: The Document Object Model is the browser object representation of the HTML document. JavaScript can query, create, update and remove nodes.

Code / example:

```
const title=document.querySelector("h1");
title.textContent="Hello";
```

125. querySelector vs getElementById?

Easy answer: getElementById specifically looks up an element by id. querySelector accepts a CSS selector and returns the first matching element.

Code / example:

```
document.getElementById("app");
document.querySelector(".card");
```

126. What does querySelectorAll return?

Easy answer: It returns a static NodeList containing all matching elements. Array methods such as map are not directly available on NodeList in the same way as arrays, so convert when needed.

Code / example:

```
const nodes=document.querySelectorAll(".card");
nodes.forEach(n=>console.log(n));
```

127. What is event bubbling?

Easy answer: After an event occurs on a target, it can propagate from the target upward through ancestor elements. This enables event delegation.

Code / example:

```
parent.addEventListener("click", e=>{
  console.log("clicked", e.target);
});
```

128. What is event capturing?

Easy answer: Capturing is the earlier propagation phase where an event travels from the document toward the target. Use the third `addEventListener` argument or options to listen in capture mode.

Code / example:

```
element.addEventListener("click", handler, {capture:true});
```

129. What is event delegation?

Easy answer: Event delegation means placing one listener on a parent and using the event target to determine which child was interacted with. It reduces listeners and works well for dynamic lists.

Code / example:

```
list.addEventListener("click", e=>{
  if(e.target.matches("button")) console.log(e.target.dataset.id);
});
```

130. localStorage vs sessionStorage?

Easy answer: `localStorage` persists until removed and is scoped to the origin. `sessionStorage` lasts for the page session/tab context. Both are string-based and accessible to JavaScript.

Code / example:

```
localStorage.setItem("theme", "dark");
sessionStorage.setItem("step", "2");
```

131. How do you store objects in localStorage?

Easy answer: Convert objects to a JSON string with `JSON.stringify` and parse it back with `JSON.parse`. Avoid storing sensitive tokens in browser storage unless you understand the security trade-offs.

Code / example:

```
localStorage.setItem("user", JSON.stringify(user));
const saved=JSON.parse(localStorage.getItem("user"));
```

132. What is debouncing?

Easy answer: Debouncing delays a function until calls stop for a chosen time. It is useful for search inputs so you do not send a request on every keystroke.

Code / example:

```
function debounce(fn,delay){
  let timer;
  return (...args)=>{
    clearTimeout(timer);
    timer=setTimeout(()=>fn(...args),delay);
  };
}
```

133. What is throttling?

Easy answer: Throttling limits execution to at most a chosen frequency. It is useful for scroll, resize and mousemove handlers.

Code / example:

```
function throttle(fn,delay){
  let last=0;
  return (...args)=>{
    const now=Date.now();
    if(now-last>=delay){ last=now; fn(...args); }
  };
}
```

134. What is memoization?

Easy answer: Memoization caches a function result for given inputs so repeated calls can reuse previous work. It helps when calculations are expensive and inputs repeat.

Code / example:

```
function memoize(fn){
  const cache=new Map();
  return x=>{ if(cache.has(x)) return cache.get(x); const v=fn(x); cache.set(x,v); return v; };
}
```

135. What is currying?

Easy answer: Currying transforms a function taking multiple arguments into a sequence of functions, each taking one argument. It can help create reusable specialized functions.

Code / example:

```
const add=a=>b=>a+b;
const add5=add(5);
add5(3); // 8
```

136. What is a prototype?

Easy answer: Objects in JavaScript can inherit properties and methods through a prototype chain. Shared methods such as Array methods are available through prototypes.

Code / example:

```
const user={name:"A"};
console.log(Object.getPrototypeOf(user));
```

137. What is prototypal inheritance?

Easy answer: An object can access properties from another object through its prototype. JavaScript classes are syntax built on this prototype mechanism.

Code / example:

```
const parent={greet(){return "hi"}};
const child=Object.create(parent);
child.greet();
```

138. What is a class in JavaScript?

Easy answer: class is syntax for creating constructor functions and prototype methods. It provides a familiar object-oriented style while still using prototypes underneath.

Code / example:

```
class User{
  constructor(name){ this.name=name; }
  greet(){ return `Hi ${this.name}`; }
}
```

139. What are modules?

Easy answer: Modules split code into reusable files. ES Modules use import/export and help organize dependencies and boundaries.

Code / example:

```
// math.js
export const add=(a,b)=>a+b;
// app.js
import {add} from "./math.js";
```

140. CommonJS vs ES Modules?

Easy answer: CommonJS uses require/module.exports and is common in older Node code. ES Modules use import/export and are the standard module syntax in modern JavaScript and Node when configured for ESM.

Code / example:

```
// CommonJS
const x=require("./x");
// ESM
import x from "./x.js";
```

141. What are generators?

Easy answer: Generator functions can pause and resume execution using yield. Calling a generator returns an iterator.

Code / example:

```
function* ids(){
  yield 1; yield 2;
}
const g=ids();
console.log(g.next());
```

142. What are iterators?

Easy answer: An iterator is an object with a next() method that returns {value, done}. for...of consumes iterables by obtaining iterators.

Code / example:

```
const iterator=[1,2][Symbol.iterator]();
iterator.next();
```

143. What are Symbol, WeakMap and WeakSet?

Easy answer: Symbol creates unique primitive values useful for non-colliding keys. WeakMap and WeakSet hold weak references to objects, allowing garbage collection when there are no other references.

Code / example:

```
const key=Symbol("id");
const wm=new WeakMap();
const obj={};
wm.set(obj,123);
```

144. What is garbage collection?

Easy answer: JavaScript engines automatically reclaim memory that is no longer reachable. You normally avoid memory leaks by removing unnecessary event listeners, timers and long-lived references.

Code / example:

```
const button=document.querySelector("button");
function handler(){}
button.addEventListener("click",handler);
// later: button.removeEventListener("click",handler);
```

145. What is tree shaking?

Easy answer: Tree shaking removes unused module exports during bundling when the tool can statically determine they are not needed. ES module syntax helps bundlers do this.

Code / example:

```
export function used(){}
export function unused(){}
```

React.js

146. What is React?

Easy answer: React is a JavaScript library for building user interfaces with reusable components and declarative rendering.

Code / example:

```
function App(){ return <h1>Hello</h1>; }
```

147. Why use React?

Easy answer: React encourages reusable components, predictable state-driven UI and a rich ecosystem. It is especially useful for applications with many interactive UI states.

Code / example:

```
function Button(){ return <button>Save</button>; }
```

148. What is a component?

Easy answer: A component is a reusable UI unit containing markup and logic. Modern React mainly uses function components and Hooks.

Code / example:

```
function UserCard({user}){ return <div>{user.name}</div>; }
```

149. Functional vs class components?

Easy answer: Function components are simpler and use Hooks for state/effects. Class components use lifecycle methods and this; they are still supported but less common in new code.

Code / example:

```
function Counter(){ const [count,setCount]=useState(0); return <button onClick={()=>setCount(count+1)}>{count}</button>; }
```

150. What is JSX?

Easy answer: JSX is syntax that lets you write HTML-like markup inside JavaScript. A build tool transforms it into JavaScript that React can use.

Code / example:

```
const el = <h1 className="title">Hello</h1>;
```

151. Why is JSX not HTML?

Easy answer: JSX looks like HTML but follows JavaScript syntax rules, such as camelCase props and JavaScript expressions inside {}. It is compiled/transformed before execution.

Code / example:

```
const name="Abhay";
const el=<h1>Hello {name}</h1>;
```

152. What is Babel?

Easy answer: Babel is a JavaScript compiler/transformation tool. In React projects it can transform JSX and modern syntax into code suitable for the chosen environment.

Code / example:

```
// JSX -> JavaScript transformation is handled by tooling
```

153. What is the Virtual DOM?

Easy answer: It is a lightweight representation of the UI that React uses as part of its rendering model. React compares rendered output and applies necessary updates to the browser DOM.

Code / example:

```
<h1>{count}</h1>
```

154. What is reconciliation?

Easy answer: Reconciliation is React comparing the previous rendered tree with the next one and determining what needs to change. Keys help React identify list items across renders.

Code / example:

```
items.map(item => <li key={item.id}>{item.name}</li>)
```

155. What are props?

Easy answer: Props are read-only inputs passed from a parent to a child component. They are the normal way to pass data and callbacks down the component tree.

Code / example:

```
function User({name}){ return <h2>{name}</h2>; }  
<User name="Abhay" />
```

156. What is state?

Easy answer: State is data managed by a component that can change over time. Updating state requests a new render so the UI can reflect the new value.

Code / example:

```
const [count, setCount]=useState(0);
```

157. Props vs state?

Easy answer: Props come from a parent and should be treated as read-only by the receiving component. State belongs to the component and can change using its update function.

Code / example:

```
function Child({name}){  
  const [open, setOpen]=useState(false);  
  return ...;  
}
```

158. What is useState?

Easy answer: useState adds local state to a function component. It returns the current state value and a setter. The setter schedules a render with the new state.

Code / example:

```
const [count, setCount]=useState(0);  
setCount(prev => prev+1);
```

159. Why should state not be mutated directly?

Easy answer: Direct mutation may not notify React correctly and can cause confusing stale UI/reference behavior. Create a new array/object and pass it to the setter.

Code / example:

```
setUser(prev => ({...prev, name:"New"}));  
setItems(prev => prev.filter(x => x.id !== id));
```

160. Why use functional state updates?

Easy answer: When the new state depends on the previous state, the functional form is safest because React may batch updates. It receives the latest state value.

Code / example:

```
setCount(prev => prev + 1);  
setCount(prev => prev + 1);
```

161. What is useEffect?

Easy answer: `useEffect` synchronizes a component with an external system such as a network request, timer, subscription or browser API. It runs after React commits the render.

Code / example:

```
useEffect(() => {
  document.title = `Count: ${count}`;
}, [count]);
```

162. What does an empty dependency array mean?

Easy answer: It tells React the effect has no reactive dependencies and should not re-run because component values changed. In development Strict Mode, React may run setup/cleanup extra times to expose bugs.

Code / example:

```
useEffect(() => { loadUser(); }, []);
```

163. What if dependency array is omitted?

Easy answer: The effect runs after every completed render. This can be correct sometimes, but it can also cause repeated work or loops if the effect updates state.

Code / example:

```
useEffect(() => { console.log("every render"); });
```

164. How do dependencies work?

Easy answer: React compares dependency values between renders. If one changes, React runs cleanup for the previous effect and then the new setup.

Code / example:

```
useEffect(() => { fetchUser(userId); }, [userId]);
```

165. What is effect cleanup?

Easy answer: The function returned from an effect is used to clean resources such as timers, event listeners and subscriptions before the effect re-runs or the component unmounts.

Code / example:

```
useEffect(() => {
  const id=setInterval(tick,1000);
  return () => clearInterval(id);
}, []);
```

166. What is `useRef`?

Easy answer: `useRef` stores a mutable value that persists across renders without causing a re-render when changed. It is also commonly used to reference DOM nodes.

Code / example:

```
const inputRef=useRef(null);
inputRef.current?.focus();
```

167. `useMemo` vs `useCallback`?

Easy answer: `useMemo` memoizes a calculated value. `useCallback` memoizes a function reference. Both should be used when they solve a measured dependency/render problem, not automatically everywhere.

Code / example:

```
const total=useMemo(()=>expensive(items),[items]);
const onSave=useCallback(()=>save(id),[id]);
```

168. What is `useContext`?

Easy answer: `useContext` reads a value supplied by a Context provider, avoiding repetitive prop passing through intermediate components.

Code / example:

```
const theme=useContext(ThemeContext);
```

169. What is a custom hook?

Easy answer: A custom Hook is a reusable function starting with `use` that combines existing Hooks and shared logic.

Code / example:

```
function useOnline(){
  const [online,setOnline]=useState(navigator.onLine);
  return online;
}
```

170. Controlled vs uncontrolled components?

Easy answer: A controlled input gets its current value from React state. An uncontrolled input lets the DOM hold the value and is often accessed through a ref. Controlled inputs are common when validation/UI state depends on the input value.

Code / example:

```
<input value={email} onChange={e=>setEmail(e.target.value)} />
```

171. How do you handle form submission in React?

Easy answer: Prevent the browser's default submit, read controlled state or FormData, validate, then call the API and handle loading/error/success states.

Code / example:

```
function submit(e){
  e.preventDefault();
  login({email,password});
}
```

172. What is React Router?

Easy answer: React Router maps URLs to React UI and gives navigation/data APIs for client-side routing.

Code / example:

```
<Route path="/jobs/:id" element={}<JobDetails /> />
```

173. Link vs NavLink?

Easy answer: Link navigates without reloading the page. NavLink also provides information about whether the link is active, useful for navigation styling.

Code / example:

```
<NavLink to="/jobs" className={({isActive}) => isActive ? "active" : ""}>Jobs</NavLink>
```

174. What is useNavigate?

Easy answer: useNavigate lets code navigate programmatically, for example after login or form submission.

Code / example:

```
const navigate=useNavigate();
navigate("/dashboard");
```

175. What is useParams?

Easy answer: useParams reads dynamic URL parameters defined by a route.

Code / example:

```
const {id}=useParams();
```

176. What are protected routes?

Easy answer: A protected route checks whether the user is authenticated and possibly authorized before rendering the page. Usually the backend must also enforce the same permissions because frontend checks are not security.

Code / example:

```
return isLoggedIn ? <Dashboard/> : <Navigate to="/login" replace />;
```

177. What is Context API?

Easy answer: Context provides a way to share values across a component tree without passing props through every level. Good examples are theme, locale and auth state.

Code / example:

```
<AuthContext.Provider value={auth}>
  <App/>
</AuthContext.Provider>
```

178. Context vs Redux?

Easy answer: Context is built into React and is good for relatively simple global values. Redux Toolkit provides a more structured state-management model, middleware/tooling and patterns useful for larger client state.

Code / example:

```
const user=useContext(AuthContext);
```

179. What is Redux?

Easy answer: Redux is a predictable state container. State is stored centrally, updated through actions/reducers, and can be inspected with developer tooling.

Code / example:

```
dispatch({type:"cart/add", payload:item});
```

180. Store, action, reducer, dispatch and selector?

Easy answer: Store holds state. An action describes what happened. A reducer calculates the next state. dispatch sends an action. A selector reads needed state.

Code / example:

```
dispatch(addItem(product));
const items=useSelector(selectItems);
```

181. What is Redux Toolkit?

Easy answer: Redux Toolkit is the recommended way to write Redux. It reduces boilerplate and provides configureStore, createSlice, createAsyncThunk and other utilities.

Code / example:

```
const slice=createSlice({name:"user",initialState, reducers:{logout:()=>null}});
```

182. What is createSlice?

Easy answer: createSlice generates action creators and a reducer from a concise slice definition. Immer lets you write update logic that looks mutable while safely producing immutable state updates.

Code / example:

```
const counterSlice=createSlice({
  name:"counter", initialState:0,
  reducers:{inc:state=>state+1}
});
```

183. What is createAsyncThunk?

Easy answer: It standardizes async action lifecycles such as pending, fulfilled and rejected. It is useful for request state managed inside Redux.

Code / example:

```
export const fetchUsers=createAsyncThunk("users/fetch", async()=>{
  const r=await fetch("/api/users"); return r.json();
});
```

184. What is RTK Query?

Easy answer: RTK Query is a data fetching and caching tool built into Redux Toolkit. It can manage requests, caching, loading/error states and invalidation with less manual code.

Code / example:

```
const {data,isLoading}=useGetJobsQuery();
```

185. RTK Query vs Axios?

Easy answer: Axios is an HTTP client. RTK Query is an API state/data-fetching solution that can use a base query internally and adds caching, invalidation and request lifecycle handling. They solve overlapping but different problems.

Code / example:

```
// RTK Query
const {data}=useGetJobsQuery();
```

186. What causes a React component to re-render?

Easy answer: A component can render again when its state changes, its parent renders and gives it new props/context, or certain external subscriptions cause updates. Rendering is not the same as always changing the real DOM.

Code / example:

```
setCount(c=>c+1); // schedules a render
```

187. What is React.memo?

Easy answer: React.memo can skip rendering a component when its props are equal according to a shallow comparison. It helps only when renders are expensive or frequent enough to matter.

Code / example:

```
const UserCard=React.memo(function UserCard({user}){ return <div>{user.name}</div>; });
```

188. What are keys and why not always use array index?

Easy answer: Keys provide stable identity for list items. Indexes can cause state/UI bugs when items are inserted, deleted or reordered because React may associate the old component state with a different item.

Code / example:

```
items.map(item => <Row key={item.id} item={item} />)
```

189. What is lazy loading/code splitting?

Easy answer: Code splitting loads some JavaScript only when it is needed. React.lazy and Suspense can load route-level components on demand, reducing initial bundle size.

Code / example:

```
const Settings=lazy(()=>import("./Settings"));
<Suspense fallback={<p>Loading...</p>}><Settings/></Suspense>
```

190. What is Suspense?

Easy answer: Suspense provides a fallback UI while certain React operations such as lazy-loaded components are waiting. It is also part of newer React patterns for data/render coordination.

Code / example:

```
<Suspense fallback={<Spinner/>}>
  <Profile/>
</Suspense>
```

191. How do you optimize React performance?

Easy answer: Measure first. Then reduce unnecessary renders, memoize where useful, split bundles, virtualize huge lists, cache server data, avoid duplicate requests and keep component/state boundaries sensible.

Code / example:

```
const filtered=useMemo(()=>items.filter(x=>x.active),[items]);
```

192. CSR vs SSR?

Easy answer: Client-side rendering builds most UI in the browser after JavaScript loads. Server-side rendering generates HTML on the server for the initial request, which can improve initial content delivery and SEO in suitable applications.

Code / example:

```
// Typical React SPA: client-side rendering
// Frameworks like Next.js can provide SSR.
```

193. SSR vs SSG?

Easy answer: SSR renders per request. SSG generates pages ahead of time, often during build. SSG is excellent for mostly static content; SSR is useful when HTML depends on request-time data.

Code / example:

```
// SSG: generate /about at build time
// SSR: render /dashboard on request
```

Node.js

194. What is Node.js?

Easy answer: Node.js is a JavaScript runtime built around the V8 engine. It lets JavaScript run outside the browser, commonly to build APIs, servers, scripts and real-time services.

Code / example:

```
import http from "node:http";
http.createServer((req,res)=>res.end("OK")).listen(5000);
```

195. Is Node.js a language?

Easy answer: No. JavaScript is the language. Node.js is the runtime environment that executes JavaScript outside the browser.

Code / example:

```
console.log(process.version);
```

196. Is Node.js a framework?

Easy answer: No. Node.js is a runtime. Frameworks/libraries such as Express, Fastify or NestJS can run on top of Node.js.

Code / example:

```
import express from "express";
```

197. What is V8?

Easy answer: V8 is a JavaScript engine developed by Google and used by Chrome and Node.js. It parses and executes JavaScript and applies runtime optimizations.

Code / example:

```
// Node executes JS using V8
console.log(1+2);
```

198. Why is Node.js good for I/O-heavy applications?

Easy answer: Node uses an event-driven, non-blocking I/O model so a JavaScript thread can keep handling other work while file/network operations complete. This is useful for APIs, chat and streaming workloads.

Code / example:

```
const data = await fs.promises.readFile("large.txt", "utf8");
```

199. Is Node.js single-threaded?

Easy answer: JavaScript execution is primarily handled by the main thread, but Node also uses the operating system and a worker pool for some operations. So saying only "Node is single-threaded" is an oversimplification.

Code / example:

```
setImmediate(()=>console.log("later"));
```

200. What is non-blocking I/O?

Easy answer: The program starts an I/O operation and does not sit idle waiting synchronously for it to finish. A callback/Promise is completed later.

Code / example:

```
fs.readFile("file.txt", "utf8", (err, data) => { console.log(data); });
console.log("can run first");
```

201. What is the Node event loop?

Easy answer: The event loop coordinates callbacks from timers, network I/O and other async sources so Node can keep progressing while I/O is in flight.

Code / example:

```
setTimeout(()=>console.log("timer"), 0);
console.log("sync");
```

202. What is npm?

Easy answer: npm is a package manager and package registry commonly used with Node.js projects. It installs dependencies and runs project scripts.

Code / example:

```
npm install express
npm run dev
```

203. package.json vs package-lock.json?

Easy answer: package.json describes the project, scripts and dependency ranges. package-lock.json records the resolved dependency tree so installs are more reproducible.

Code / example:

```
{ "scripts": { "dev": "node server.js" } }
```

204. What is node_modules?

Easy answer: It is the directory where installed dependencies are stored locally. It is normally not committed to Git because it can be recreated from package metadata.

Code / example:

```
npm install
# creates node_modules/
```

205. What is npm install vs npm install --save-dev?

Easy answer: Normal install adds a dependency needed by the application. --save-dev adds a development dependency such as a test runner, linter or build tool.

Code / example:

```
npm install express
npm install -D nodemon
```

206. What are environment variables?

Easy answer: They provide configuration outside source code, such as PORT, database URLs and secrets. They make the same code usable in different environments.

Code / example:

```
const port = process.env.PORT || 5000;
```

207. CommonJS vs ESM in Node?

Easy answer: CommonJS uses require/module.exports; ES Modules use import/export. Choose one module system consistently per project/configuration.

Code / example:

```
// ESM
import express from "express";
// CommonJS
const express = require("express");
```

208. What is require?

Easy answer: require loads a CommonJS module and returns the value exported by that module.

Code / example:

```
const bcrypt=require("bcryptjs");
```

209. What is module.exports?

Easy answer: module.exports defines the value that a CommonJS module exposes to other files.

Code / example:

```
module.exports = { add, sub };
```

210. What are streams?

Easy answer: Streams process data incrementally rather than loading everything into memory at once. They are useful for large files, uploads/downloads and network data.

Code / example:

```
fs.createReadStream("big.zip").pipe(res);
```

211. What is a Buffer?

Easy answer: A Buffer represents raw binary data in Node. It is used for files, network protocols and other binary operations.

Code / example:

```
const buf = Buffer.from("hello");
console.log(buf);
```

212. What is the fs module?

Easy answer: fs provides filesystem APIs such as reading, writing, renaming and deleting files. Prefer asynchronous APIs for server request paths when practical.

Code / example:

```
import { readFile } from "node:fs/promises";
const text=await readFile("a.txt","utf8");
```

213. sync vs async filesystem APIs?

Easy answer: Synchronous APIs block the JavaScript execution path until they finish. Async APIs let Node continue handling other work and notify you later. Async is usually preferred in servers.

Code / example:

```
const data = readFileSync("a.txt","utf8");
const data2 = await readFile("a.txt","utf8");
```

214. What is a Node error-first callback?

Easy answer: Traditional Node callbacks often receive the error as the first argument and the result as the second argument.

Code / example:

```
fs.readFile("a.txt","utf8",(err,data)=>{
  if(err) return console.error(err);
  console.log(data);
});
```

215. What are worker threads?

Easy answer: Worker threads allow CPU-heavy JavaScript work to run in separate threads. They are useful when CPU computation would otherwise block the event loop.

Code / example:

```
import { Worker } from "node:worker_threads";
const worker=new Worker("./worker.js");
```

216. What is a child process?

Easy answer: Node can start external processes using child_process. This is useful for executing system commands or separate programs, but inputs must be handled safely.

Code / example:

```
import { exec } from "node:child_process";
exec("node -v", console.log);
```

217. What is process.env.NODE_ENV?

Easy answer: It is a common convention for identifying an environment such as development or production. It is not a special security boundary by itself.

Code / example:

```
if(process.env.NODE_ENV === "production"){
  console.log("production mode");
}
```

218. How do you handle uncaught errors in Node?

Easy answer: Handle expected errors at the appropriate boundary, log useful details, and use process-level handlers only as a last line of defense. For fatal states, restarting the process under a supervisor is usually safer than continuing in an unknown state.

Code / example:

```
process.on("uncaughtException", err => {
  console.error(err);
  process.exit(1);
});
```

Express.js

219. What is Express.js?

Easy answer: Express is a lightweight web framework for Node.js used to build HTTP servers and APIs with routing and middleware.

Code / example:

```
import express from "express";
const app=express();
app.listen(5000);
```

220. Why use Express with Node?

Easy answer: Node gives the runtime and low-level HTTP APIs. Express provides convenient routing, middleware composition and request/response helpers, making API development faster.

Code / example:

```
app.get("/users", (req,res)=>res.json([]));
```

221. How do you create an Express server?

Easy answer: Create an Express application, register middleware/routes, and call app.listen with a port.

Code / example:

```
const app=express();
app.use(express.json());
app.get("/",(req,res)=>res.send("Hello"));
app.listen(5000);
```

222. What is middleware?

Easy answer: Middleware is a function that runs during the request-response lifecycle. It can read/modify req/res, stop the request by sending a response, or call next() to continue.

Code / example:

```
app.use((req,res,next)=>{
  console.log(req.method,req.url);
  next();
});
```

223. Types of Express middleware?

Easy answer: Common categories are application-level, router-level, built-in, third-party and error-handling middleware.

Code / example:

```
app.use(express.json());
app.use(cors());
```

224. What does next() do?

Easy answer: next passes control to the next matching middleware or route handler. If middleware neither sends a response nor calls next, the request can hang.

Code / example:

```
app.use((req,res,next)=>{ req.start=Date.now(); next(); });
```

225. What is routing?

Easy answer: Routing maps an HTTP method and path to a handler. Express supports route parameters, routers and multiple handlers.

Code / example:

```
app.get("/users", getUsers);
app.post("/users", createUser);
```

226. What is a route parameter?

Easy answer: A route parameter is a dynamic segment in a URL, such as :id. It is read from req.params.

Code / example:

```
app.get("/users/:id", (req, res) => {
  console.log(req.params.id);
});
```

227. What is a query parameter?

Easy answer: Query parameters appear after ? and are commonly used for filtering, sorting, searching and pagination. They are available in req.query.

Code / example:

```
GET /jobs?city=Delhi&page=2
req.query.city // Delhi
```

228. GET vs POST in APIs?

Easy answer: GET is normally safe/idempotent retrieval. POST usually submits data to create a resource or trigger an action and is not automatically idempotent.

Code / example:

```
app.get("/jobs", listJobs);
app.post("/jobs", createJob);
```

229. PUT vs PATCH?

Easy answer: PUT is commonly used for full replacement of a resource representation. PATCH is for partial modification. Actual API semantics should be documented and consistent.

Code / example:

```
app.put("/users/:id", replaceUser);
app.patch("/users/:id", updateUser);
```

230. What is a controller?

Easy answer: A controller handles HTTP details: read input, call business logic, and return a response. Keeping controllers thin makes code easier to test and maintain.

Code / example:

```
export async function getUser(req, res, next) {
  const user = await userService.get(req.params.id);
  res.json(user);
}
```

231. Why separate routes and controllers?

Easy answer: Routes describe which endpoint maps to which handler. Controllers hold request-level logic. Separation prevents one huge server file and makes testing and maintenance easier.

Code / example:

```
router.get("/users", userController.list);
```

232. What is a service layer?

Easy answer: A service layer contains business rules and workflows that should not be tightly coupled to Express request/response objects.

Code / example:

```
export async function createJob(data, userId) {
  // business rules
  return Job.create({...data, employer: userId});
}
```

233. What is MVC?

Easy answer: Model represents data, View represents UI, Controller handles requests and coordinates work. In a JSON API, the response body plays the role of the output instead of server-rendered HTML.

Code / example:

```
Route -> Controller -> Model -> JSON response
```

234. What is centralized error handling?

Easy answer: It means errors are sent through one consistent error middleware so APIs return predictable status codes/messages and logging is centralized.

Code / example:

```
app.use((err, req, res, next) => {
  res.status(err.status || 500).json({message: err.message});
});
```

235. How do you parse JSON body?

Easy answer: Use Express built-in `express.json` middleware. Without it, `req.body` will usually be undefined for JSON requests.

Code / example:

```
app.use(express.json());
```

236. How do you serve static files?

Easy answer: Use `express.static` to expose a directory. Avoid exposing secrets or private files by placing only intended public assets there.

Code / example:

```
app.use("/uploads", express.static("uploads"));
```

237. What is a router in Express?

Easy answer: `express.Router` creates a modular route handler so routes can be grouped by resource/feature and mounted under a common path.

Code / example:

```
const router = express.Router();
router.get("/", list);
app.use("/api/jobs", router);
```

238. How do you handle async errors?

Easy answer: Wrap async handlers with a reusable async wrapper or use an Express setup/version that forwards rejected async handlers to error middleware. Always ensure failures reach the central error handler.

Code / example:

```
const asyncHandler = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
```

239. What is CORS middleware?

Easy answer: It adds appropriate cross-origin response headers so browsers can allow requests from configured origins. CORS does not protect the API from non-browser clients.

Code / example:

```
app.use(cors({origin: "http://localhost:5173", credentials: true}));
```

240. What is Helmet?

Easy answer: Helmet is middleware that sets several security-related HTTP headers. It is useful as one part of a broader security strategy.

Code / example:

```
app.use(helmet());
```

241. How do you validate request data?

Easy answer: Validate body, params and query values before business logic. Reject unexpected shapes or invalid values and return a clear client error.

Code / example:

```
if(typeof req.body.email !== "string")
  return res.status(400).json({message: "Invalid email"});
```

242. What is a 404 route handler?

Easy answer: It is middleware placed after all real routes that returns Not Found when no route matched.

Code / example:

```
app.use((req, res) => res.status(404).json({message: "Route not found"}));
```

243. What is the difference between 401 and 403 in Express?

Easy answer: 401 means the request lacks valid authentication credentials. 403 means the server understood who the user is but refuses access because of permissions.

Code / example:

```
if(!req.user) return res.status(401).json({message: "Login required"});
if(req.user.role !== "admin") return res.status(403).json({message: "Forbidden"});
```

MongoDB & Mongoose

244. What is MongoDB?

Easy answer: MongoDB is a NoSQL document database. It stores BSON documents inside collections rather than rows inside relational tables.

Code / example:

```
{ name: "Abhay", skills: ["React", "Node"] }
```

245. What is NoSQL?

Easy answer: NoSQL is a broad family of databases that do not all use the traditional relational table model. MongoDB uses documents and flexible schemas.

Code / example:

```
db.users.find({role:"candidate"});
```

246. Database vs collection vs document?

Easy answer: A database contains collections; a collection contains documents; a document is an individual record. This roughly corresponds to database/table/row, but MongoDB is not a relational table model.

Code / example:

```
db.users.insertOne({name:"Abhay"});
```

247. What is BSON?

Easy answer: BSON is MongoDB's binary representation of JSON-like data. It supports extra types such as ObjectId and Date.

Code / example:

```
{ createdAt: new Date(), userId: ObjectId("...") }
```

248. What is _id?

Easy answer: MongoDB documents normally have a unique _id field. If you do not provide one, MongoDB can generate one automatically.

Code / example:

```
{ _id: ObjectId("..."), name:"Abhay" }
```

249. What is ObjectId?

Easy answer: ObjectId is a common MongoDB identifier type. It is compact and contains time-related and unique components; in applications you should validate strings before converting them when appropriate.

Code / example:

```
new mongoose.Types.ObjectId(id)
```

250. What is CRUD?

Easy answer: CRUD means Create, Read, Update and Delete. These are the basic operations you perform on stored data.

Code / example:

```
create -> insert
read -> find
update -> updateOne
delete -> deleteOne
```

251. How do you insert a document?

Easy answer: Use insertOne/insertMany in the Mongo shell/driver or Model.create in Mongoose.

Code / example:

```
await User.create({name:"Abhay",email:"a@example.com"});
```

252. How do you read documents?

Easy answer: Use find for many documents and findOne/findById for one. Add filters, projection and pagination to avoid retrieving unnecessary data.

Code / example:

```
const users=await User.find({role:"candidate"}).select("name email");
```

253. How do you update documents?

Easy answer: Update operators such as \$set and \$inc modify specific fields. In Mongoose, updateOne/findByIdAndUpdate are common methods.

Code / example:

```
await User.updateOne({email},{ $set:{name:"New"}});
```

254. How do you delete documents?

Easy answer: Use deleteOne/deleteMany or Mongoose equivalents. In production systems, consider whether soft delete is more appropriate for audit/recovery needs.

Code / example:

```
await User.deleteOne({_id:userId});
```

255. What are MongoDB query operators?

Easy answer: Common operators include \$eq, \$ne, \$gt, \$gte, \$lt, \$lte, \$in, \$nin, \$or, \$and and \$exists. Update operators include \$set, \$inc, \$push and \$pull.

Code / example:

```
Job.find({salary:{ $gte:50000},skills:{ $in:["React"]}});
```

256. What is an index?

Easy answer: An index is a data structure that helps MongoDB locate matching documents efficiently. Indexes can greatly improve reads but use memory/storage and add work to writes.

Code / example:

```
db.users.createIndex({email:1},{unique:true});
```

257. What is a unique index?

Easy answer: It prevents multiple documents from having the same indexed value (subject to MongoDB uniqueness rules, such as how missing/null values are represented).

Code / example:

```
db.users.createIndex({email:1},{unique:true});
```

258. What is a compound index?

Easy answer: It indexes multiple fields together. Field order matters and should match common filter/sort patterns.

Code / example:

```
db.jobs.createIndex({city:1,createdAt:-1});
```

259. What is aggregation?

Easy answer: Aggregation processes documents through stages to filter, transform, group and join data. It is useful for analytics and complex read models.

Code / example:

```
db.orders.aggregate([
  { $match:{status:"completed"}},
  { $group:{_id:"$userId",total:{ $sum:"$amount"}}}
]);
```

260. What is \$match?

Easy answer: \$match filters documents early in an aggregation pipeline. Putting selective filtering early can reduce later work.

Code / example:

```
{ $match:{status:"active"}}
```

261. What is \$group?

Easy answer: \$group combines documents by a key and computes accumulations such as \$sum, \$avg, \$min and \$max.

Code / example:

```
{ $group:{_id:"$companyId",count:{ $sum:1}}}
```

262. What is \$sort?

Easy answer: \$sort orders documents by one or more fields. Indexes can help certain sorting patterns.

Code / example:

```
{ $sort:{createdAt:-1}}
```

263. What is \$project?

Easy answer: \$project controls the shape of documents in the pipeline and can compute new fields.

Code / example:

```
{ $project:{name:1,email:1,_id:0}}
```

264. What is \$lookup?

Easy answer: \$lookup performs a join-like operation with another collection. It is useful but can be expensive, so schema design and indexes matter.

Code / example:

```
{ $lookup: { from: "users", localField: "userId", foreignField: "_id", as: "user" } }
```

265. What is Mongoose?

Easy answer: Mongoose is an ODM for MongoDB in Node.js. It provides schemas, models, validation, middleware and convenient query APIs.

Code / example:

```
const User = mongoose.model("User", userSchema);
```

266. What is a Mongoose schema?

Easy answer: A schema describes fields, types, validation, defaults and optional middleware for a model.

Code / example:

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true }
});
```

267. What is a Mongoose model?

Easy answer: A model is the API used to create and query documents associated with a collection.

Code / example:

```
const User = mongoose.model("User", userSchema);
const users = await User.find();
```

268. What is validation in Mongoose?

Easy answer: Schema validators can reject invalid data before saving. Still validate API input at the request boundary because not every operation necessarily passes through the same model path and validation is not authorization.

Code / example:

```
email: { type: String, required: true }
age: { type: Number, min: 18 }
```

269. What are Mongoose hooks/middleware?

Easy answer: Hooks run before/after certain model operations, such as saving. They can be useful for things like password hashing when carefully designed.

Code / example:

```
userSchema.pre("save", async function(next){
  // hash password if changed
  next();
});
```

270. What is populate?

Easy answer: populate replaces referenced IDs with related documents through additional database work. It is convenient for API responses, but you should select only fields you need.

Code / example:

```
Post.find().populate("author", "name email");
```

271. Embedding vs referencing?

Easy answer: Embed when data is small, bounded and usually read with the parent. Reference when related data is large, shared, or has an independent lifecycle.

Code / example:

```
// embedded
{ address: { city: "Delhi" } }
// referenced
{ employer: ObjectId("...") }
```

272. How do you model one-to-many?

Easy answer: You can embed a bounded child array or reference child documents to a parent ID. The choice depends on document size, update patterns and access patterns.

Code / example:

```
// Job has employer reference
{ title: "Developer", employer: ObjectId("...") }
```

273. How do you model many-to-many?

Easy answer: Commonly store arrays of references on one side or a separate relationship collection when the relationship itself has attributes or can grow very large.

Code / example:

```
{studentIds:[ObjectId("...")], courseId:ObjectId("...")}
```

274. How do you implement pagination in MongoDB?

Easy answer: A simple offset approach uses skip/limit, but for large/high-volume datasets cursor/range pagination can scale better and provide more stable performance. Always sort by a stable indexed field.

Code / example:

```
Job.find({createdAt:{$lt:lastSeen}})
  .sort({createdAt:-1})
  .limit(20);
```

Backend Concepts & Security

275. What is a backend?

Easy answer: The backend is the server-side part of an application. It receives requests, applies business rules, authenticates users, reads/writes data and returns responses.

Code / example:

```
Client -> API -> Business logic -> Database
```

276. What is HTTP?

Easy answer: HTTP is a request-response protocol used for communication between clients and servers. APIs commonly use HTTP methods, headers, bodies and status codes.

Code / example:

```
GET /api/jobs
POST /api/jobs
```

277. HTTP vs HTTPS?

Easy answer: HTTPS is HTTP over TLS. It provides encryption and server authentication for data in transit, helping prevent eavesdropping and tampering.

Code / example:

```
https://example.com/api/users
```

278. What is an HTTP request?

Easy answer: A request contains a method, URL, headers, and optionally a body. It communicates what the client wants the server to do.

Code / example:

```
POST /api/login
Content-Type: application/json

{"email":"a@x.com","password":"..."}
```

279. What is an HTTP response?

Easy answer: A response contains a status code, headers and optionally a body. The status code communicates whether the request succeeded and why it failed.

Code / example:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"message":"ok"}
```

280. What are headers?

Easy answer: Headers carry metadata such as content type, authorization, caching, cookies and CORS policy.

Code / example:

```
Authorization: Bearer <token>
Content-Type: application/json
```

281. What is a request body?

Easy answer: The body contains submitted data, such as JSON in a POST/PATCH request. Its format is described by headers such as Content-Type.

Code / example:

```
{ "title":"Frontend Developer" }
```

282. What are HTTP methods?

Easy answer: GET retrieves, POST creates/submits, PUT replaces, PATCH partially updates, and DELETE removes. These are conventions and should be used consistently in an API.

Code / example:

```
GET /jobs
POST /jobs
PATCH /jobs/1
DELETE /jobs/1
```

283. What are common HTTP status codes?

Easy answer: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized (missing/invalid auth), 403 Forbidden (not allowed), 404 Not Found, 409 Conflict, 422 Unprocessable Content, 500 Internal Server Error.

Code / example:

```
res.status(201).json(job);
```

284. What is REST?

Easy answer: REST is an architectural style. A RESTful API commonly models resources, uses HTTP semantics, remains stateless between requests, and returns representations such as JSON.

Code / example:

```
GET /api/users/123
```

285. What does stateless mean?

Easy answer: The server does not rely on conversational request state being remembered between requests. Each request carries the information required for that operation, often through a token/session cookie plus server-side state as needed.

Code / example:

```
Authorization: Bearer <token>
```

286. REST vs SOAP?

Easy answer: REST is an architectural style usually using HTTP and representations such as JSON. SOAP is a protocol with a formal XML-based messaging model. Modern web APIs commonly use REST-style HTTP APIs, but the right choice depends on system requirements.

Code / example:

```
// REST
GET /users/1
// SOAP uses XML envelopes
```

287. Authentication vs authorization?

Easy answer: Authentication answers "Who are you?" Authorization answers "What are you allowed to do?" They should be separate checks.

Code / example:

```
if(!req.user) return res.sendStatus(401);
if(req.user.role!=="employer") return res.sendStatus(403);
```

288. What is JWT?

Easy answer: JWT is a signed token format often used for authentication. It has header, payload and signature. Its payload is encoded, not encrypted, so do not put secrets in it.

Code / example:

```
const token=jwt.sign({userId:user._id},process.env.JWT_SECRET,{expiresIn:"15m"});
```

289. What are the three parts of JWT?

Easy answer: Header describes the token type/algorithm, payload contains claims, and signature proves the token was signed with the expected secret/key.

Code / example:

```
header.payload.signature
```

290. Why not put passwords in JWT payload?

Easy answer: JWT payloads are normally readable by whoever possesses the token because encoding is not encryption. Passwords and other secrets should never be put there.

Code / example:

```
// good
{ userId:"123", role:"candidate" }
// bad
```

```
{ password:"12345" }
```

291. JWT authentication flow?

Easy answer: Login verifies credentials, then the server signs a token or creates a session. The client sends credentials on protected requests, middleware verifies them, then authorization checks permissions.

Code / example:

```
POST /login -> token -> GET /profile + Authorization header
```

292. Access token vs refresh token?

Easy answer: An access token is short-lived and used for API access. A refresh token is longer-lived and used to obtain new access tokens. Secure storage and rotation/revocation strategy matter.

Code / example:

```
access: 15m  
refresh: longer-lived + rotation
```

293. What is hashing?

Easy answer: Hashing transforms data into a one-way digest. Passwords should be stored using a password hashing algorithm such as bcrypt or Argon2, not encrypted with a reversible key.

Code / example:

```
const hash=await bcrypt.hash(password,10);
```

294. Why bcrypt/Argon2?

Easy answer: They are designed for password hashing and intentionally expensive, which makes large-scale guessing attacks harder. They also use salts so identical passwords do not produce identical stored hashes.

Code / example:

```
const ok=await bcrypt.compare(password,hash);
```

295. What is CORS?

Easy answer: Cross-Origin Resource Sharing is a browser mechanism controlling whether a web page can make certain requests to another origin. The server opts into allowed origins/methods/headers using response headers.

Code / example:

```
app.use(cors({origin:"https://myfrontend.com",credentials:true}));
```

296. Cookie vs localStorage?

Easy answer: Cookies can be automatically sent with matching HTTP requests and can be marked HttpOnly/Secure/SameSite. localStorage is accessible to JavaScript and is not automatically attached to requests. Choose based on threat model and architecture.

Code / example:

```
res.cookie("sid",sessionId,{httpOnly:true,secure:true,sameSite:"lax"});
```

297. What is an HTTP-only cookie?

Easy answer: An HttpOnly cookie cannot be read by JavaScript running in the page, reducing exposure to token theft through direct script access. It can still be sent by the browser according to cookie rules.

Code / example:

```
res.cookie("token",token,{httpOnly:true});
```

298. What is XSS?

Easy answer: Cross-site scripting is when attacker-controlled content executes as script in a victim browser. Use safe rendering/escaping, avoid dangerous HTML injection, validate untrusted data appropriately and use security headers.

Code / example:

```
element.textContent = userInput; // safer than innerHTML for plain text
```

299. What is CSRF?

Easy answer: Cross-site request forgery tricks a browser into sending an authenticated request that the user did not intend. SameSite cookies, CSRF tokens and origin checks are common defenses depending on architecture.

Code / example:

```
// Example defense: verify CSRF token for state-changing requests
```

300. What is NoSQL injection?

Easy answer: It occurs when untrusted input can alter the structure of a MongoDB query. Validate request shapes/types and avoid passing arbitrary objects as query filters.

Code / example:

```
// Do not blindly use req.body.filter as a Mongo query
const email = String(req.body.email);
```

301. What is rate limiting?

Easy answer: Rate limiting controls how many requests a client can make in a time window. It helps protect login, password reset, search and expensive endpoints from abuse.

Code / example:

```
app.use(rateLimit({windowMs:60_000,max:100}));
```

302. What is input validation?

Easy answer: Validation checks types, required fields, formats, ranges and business constraints before processing. Server-side validation is mandatory even if the frontend validates.

Code / example:

```
if(!/^[^@]+@^[^@]+$/ .test(email)) return res.status(422).json({message:"Invalid email"});
```

303. What is pagination?

Easy answer: Pagination limits how many records an API returns at once. It reduces response size and database work and improves UI performance.

Code / example:

```
GET /jobs?page=2&limit=20
```

304. What is caching?

Easy answer: Caching stores reusable data closer to where it is needed. HTTP caches, in-memory caches and Redis can reduce database/network work when the data is safe to reuse.

Code / example:

```
const cached=await redis.get(`job:${id}`);
```

305. What is a reverse proxy?

Easy answer: A reverse proxy sits in front of backend servers and can handle TLS termination, routing, compression, caching, rate limiting and load balancing.

Code / example:

```
Client -> Nginx -> Node/Express
```

306. What is load balancing?

Easy answer: A load balancer distributes requests across multiple server instances. This helps scale traffic and improve availability if configured with health checks.

Code / example:

```
LB -> API instance 1
    -> API instance 2
```

307. What is a database transaction?

Easy answer: A transaction groups multiple database operations into an all-or-nothing unit when the database supports it. It is useful when partial success would leave inconsistent data.

Code / example:

```
session.startTransaction();
// operations
await session.commitTransaction();
```

308. What are ACID properties?

Easy answer: Atomicity means all-or-nothing, Consistency preserves rules, Isolation controls concurrent transaction interaction, Durability means committed data survives failures.

Code / example:

```
// MongoDB supports transactions for suitable deployment configurations
```

MERN Full-Stack

309. What does MERN stand for?

Easy answer: MongoDB, Express, React and Node.js. React handles the UI, Node/Express commonly handle the API, and MongoDB stores document data.

Code / example:

```
React -> Express/Node -> MongoDB
```

310. Why is MERN popular?

Easy answer: It lets teams use JavaScript across frontend and backend and has a large ecosystem. It is productive for many web applications, especially when document-oriented data and API-driven UI fit the problem.

Code / example:

```
React + Node + MongoDB
```

311. How does React communicate with the backend?

Easy answer: It sends HTTP requests using fetch, Axios or a data-fetching tool. The backend validates the request, performs business logic and returns an HTTP response, commonly JSON.

Code / example:

```
const res=await fetch("/api/jobs");
const jobs=await res.json();
```

312. How does Node communicate with MongoDB?

Easy answer: Node uses the MongoDB driver or an ODM such as Mongoose to connect and issue queries/updates.

Code / example:

```
const jobs=await Job.find({active:true});
```

313. Explain a complete login flow.

Easy answer: The user submits email/password. The server validates the input, finds the user, compares the password hash, then creates an auth token/session and sends it to the client. The client uses that credential for protected requests.

Code / example:

```
POST /api/auth/login
-> verify
-> create token/session
-> return result
```

314. Explain registration flow.

Easy answer: Validate input, ensure uniqueness/business rules, hash the password, create the user, and return a safe response. Never return the plaintext password or password hash unnecessarily.

Code / example:

```
const hash=await bcrypt.hash(password,12);
const user=await User.create({email,password:hash});
```

315. Explain a protected API flow.

Easy answer: The request carries a token/cookie. Authentication middleware verifies it and attaches user identity. Authorization checks role/ownership, then the controller/service performs the operation.

Code / example:

```
Request -> auth -> authorization -> controller -> DB
```

316. How should frontend and backend environment variables be handled?

Easy answer: Keep URLs/configuration outside source code. Never expose server secrets to the browser. Frontend build tools may expose only variables explicitly intended for client use.

Code / example:

```
SERVER: JWT_SECRET=...
CLIENT: VITE_API_URL=https://api.example.com
```

317. How do you handle CORS in development and production?

Easy answer: Allow the development frontend origin locally, and explicitly configure the real production origin in production. Avoid wildcard origins when credentials are used.

Code / example:

```
cors({origin:["http://localhost:5173","https://app.example.com"],credentials:true})
```

318. How do you structure a MERN backend?

Easy answer: A common structure is config, models, routes, controllers, services, middleware, validators and utilities. The exact structure can vary; the goal is clear responsibility boundaries.

Code / example:

```
src/
  routes/
  controllers/
  services/
  models/
  middleware/
```

319. How do you handle errors across MERN?

Easy answer: Backend should return consistent status codes and error payloads. Frontend should handle loading, success and error states and show useful user messages without leaking server internals.

Code / example:

```
try { await api(); } catch(err) { setError(err.message); }
```

320. How do you handle loading states?

Easy answer: Track loading explicitly for API actions and prevent duplicate submissions where appropriate. In data libraries, use their loading state instead of inventing duplicate state.

Code / example:

```
const [loading, setLoading]=useState(false);
```

321. How do you implement role-based access?

Easy answer: Store a role such as candidate/employer, authenticate the user, then enforce role/permission checks on the backend. Hide UI options in React for UX, but never depend on hiding them for security.

Code / example:

```
if(req.user.role !== "employer") return res.sendStatus(403);
```

322. How do you deploy MERN?

Easy answer: Build/deploy the frontend, deploy the Node/Express API, configure environment variables, allow the server to access MongoDB, configure CORS and HTTPS, then test production endpoints and logs.

Code / example:

```
Frontend -> CDN/host  
Backend -> server/platform  
DB -> managed MongoDB
```

323. What happens if MongoDB is down?

Easy answer: Database calls fail or time out. The backend should handle errors, log the incident, return an appropriate server error, and use retries/health checks only where they make sense.

Code / example:

```
try { await User.find(); } catch(e) { res.status(503).json({message:"Service unavailable"}); }
```

324. How would you scale a MERN app?

Easy answer: Keep API instances stateless, add load balancing, optimize DB indexes/queries, add caching, paginate, move long tasks to queues, use object storage/CDNs for files and monitor the system.

Code / example:

```
LB -> API1/API2 -> MongoDB  
    \-> Redis
```

325. Where should authentication state be stored?

Easy answer: There is no single answer. Secure HttpOnly cookies are often attractive for browser sessions because JavaScript cannot directly read them. Token-in-storage approaches have different trade-offs and require careful XSS protection.

Code / example:

```
res.cookie("session",id,{httpOnly:true,secure:true,sameSite:"lax"});
```

326. How does a MERN chat app use WebSockets?

Easy answer: The React client opens a socket connection. Node/Socket.IO keeps the connection and broadcasts events such as message:new to the relevant users/rooms. Messages can also be persisted in MongoDB.

Code / example:

```
socket.on("message:send", async data => {  
  await Message.create(data);  
  io.to(data.roomId).emit("message:new",data);  
});
```

327. How do you prevent duplicate records?

Easy answer: Use server-side uniqueness constraints where possible, check existing state, and design idempotent operations when retries can happen. A frontend-only check is not enough.

Code / example:

```
await User.createIndexes([{key:{email:1},unique:true}]);
```

Project & System Design Questions

328. How should you explain a project in an interview?

Easy answer: Use a fixed story: problem -> users -> major features -> stack -> architecture -> database -> authentication -> APIs -> hardest challenge -> solution -> deployment -> future improvements. Keep the first explanation around 60-120 seconds.

Code / example:

```
Problem -> Features -> Architecture -> DB -> Auth -> APIs -> Challenges -> Deployment
```

329. Why did you choose MERN for your project?

Easy answer: A good answer connects the stack to the problem: one primary language across frontend/backend, reusable React UI, productive Express/Node APIs and MongoDB when document-style data fits the use case.

Code / example:

```
React UI -> Node/Express API -> MongoDB
```

330. Why MongoDB instead of SQL?

Easy answer: Say what your application needs. MongoDB is convenient for JSON-like documents and flexible schemas; SQL may be better when complex relational constraints and joins are central. Avoid saying one is always better.

Code / example:

```
Job document -> employer reference -> application documents
```

331. How did you design your database?

Easy answer: Start from entities and access patterns, then choose embedding/references and indexes. In a job portal, likely entities include users, jobs and applications with ownership/role relationships.

Code / example:

```
User 1---* Job
User 1---* Application
Job 1---* Application
```

332. Explain one API end-to-end.

Easy answer: Say method + endpoint, request input, auth, validation, controller/service, DB operation, status code and response. This proves you understand the whole path.

Code / example:

```
POST /api/jobs
Body -> validate -> auth -> create Job -> 201 JSON
```

333. How did you protect routes?

Easy answer: Use authentication middleware to verify the session/token and attach the user ID to the request. Then check authorization such as role or record ownership before changing data.

Code / example:

```
router.post("/jobs", auth, requireEmployer, createJob);
```

334. How did you secure passwords?

Easy answer: Hash passwords with bcrypt/Argon2 before storing them. Compare submitted passwords with the hash during login. Never store or log plaintext passwords.

Code / example:

```
const hash=await bcrypt.hash(password,12);
const ok=await bcrypt.compare(password,hash);
```

335. How did you validate input?

Easy answer: Validate at the API boundary and again where domain rules require it. Reject malformed/unexpected data before database writes.

Code / example:

```
if(!title?.trim()) return res.status(422).json({message:"Title required"});
```

336. How did you handle errors?

Easy answer: Use try/catch or async error wrappers to send failures to central middleware. Return safe client messages while logging technical details on the server.

Code / example:

```
next(new AppError("Job not found",404));
```

337. What was the hardest bug in your project?

Easy answer: Use a real STAR-style answer: symptom, investigation, root cause, fix, verification and what you learned. Avoid saying simply "there were no bugs."

Code / example:

```
// Example root cause: wrong request property
req.id // wrong
req.userId // correct
```

338. How do you debug an API that works in Postman but not in React?

Easy answer: Check API URL, HTTP method, request body, headers, credentials, CORS, environment variables and browser Network tab. If Postman works, browser-only rules such as CORS become especially likely.

Code / example:

```
fetch(`${import.meta.env.VITE_API_URL}/api/jobs`, {
  headers: {Authorization: `Bearer ${token}`}
});
```

339. How do you prevent unauthorized employers/candidates?

Easy answer: Enforce authorization on the backend. Example: only employers can create jobs; candidates can apply; ownership checks prevent editing another employer's job.

Code / example:

```
if(req.user.role!=="employer") return res.sendStatus(403);
```

340. How would you implement pagination for jobs?

Easy answer: Use limit plus page/offset for simple use cases, or cursor/range pagination for high volume. Sort by a stable indexed field such as createdAt plus _id.

Code / example:

```
GET /api/jobs?limit=20&cursor=...
```

341. How would you implement search and filters?

Easy answer: Normalize the request query, build safe Mongo filters, index common exact/range filters and use text/search features appropriate to the scale. Never accept raw database query objects from users.

Code / example:

```
Job.find({city,experience:{$lte:maxExperience}}).limit(20);
```

342. How would you improve project performance?

Easy answer: Measure first. Then optimize slow queries/indexes, reduce payloads, paginate, cache repeated reads, avoid duplicate React requests, lazy-load heavy UI and move long work to background jobs.

Code / example:

```
const jobs=await Job.find(filter).select("title company city").limit(20);
```

343. How would you handle 100,000 users?

Easy answer: Use stateless API instances behind a load balancer, indexes, pagination, caching, queues for expensive tasks, monitoring, rate limiting and carefully designed DB access. The exact architecture depends on traffic shape.

Code / example:

```
LB -> API x N -> cache -> Mongo/other services
```

344. How would you improve authentication?

Easy answer: Use secure cookie/session or carefully designed short-lived access + refresh tokens, rotate/revoke where needed, hash passwords, rate-limit login, validate input and monitor suspicious activity.

Code / example:

```
POST /login -> short access -> refresh/rotation strategy
```

345. How would you test your backend?

Easy answer: Unit test business logic, integration test API/database behavior and end-to-end test key user flows. Test success, validation, authentication, authorization and failure cases.

Code / example:

```
expect(res.status).toBe(201);
expect(res.body.title).toBe("Developer");
```

346. How would you improve your React frontend?

Easy answer: Use component boundaries, reusable hooks, stable data fetching, route-level lazy loading, accessible forms, error states, responsive design and measured performance optimizations.

Code / example:

```
const JobDetails=lazy(()=>import("./JobDetails"));
```

347. How would you store uploaded resumes/images?

Easy answer: For production, large files are commonly stored in object storage and the database keeps metadata/URLs. Validate file type/size and enforce access controls.

Code / example:

```
{userId, key:"resumes/abc.pdf", size, mimeType}
```

348. What would you improve if you had more time?

Easy answer: Choose concrete improvements: stronger tests, validation, rate limiting, refresh-token strategy, search quality, pagination, better UI feedback, observability, background jobs or deployment automation.

Code / example:

```
// Prioritize improvements by user impact and risk.
```

Real Interview Scenarios

349. Login returns 401. What do you check?

Easy answer: Check whether the request includes credentials, whether the token/cookie is present and valid, JWT secret matches, expiration is correct, and middleware extracts the credential from the right place. Also confirm the frontend is calling the correct production API.

Code / example:

```
Authorization: Bearer ${token}  
// or cookie/session depending on design
```

350. MongoDB query is slow. What do you check?

Easy answer: Inspect the query, indexes and execution plan, returned document size, sort, pagination and aggregation stages. Add/select indexes based on real access patterns rather than indexing everything.

Code / example:

```
db.jobs.explain("executionStats").find({city:"Delhi"});
```

351. React page keeps making API requests. What could be wrong?

Easy answer: Common causes include useEffect without a dependency array, unstable dependencies, state updates triggering the effect again, remounting, or a data-fetching hook configured incorrectly.

Code / example:

```
useEffect(()=>{ load(); }, []); // one initial run per mount (subject to Strict Mode dev behavior)
```

352. Frontend and backend are on different domains and request fails in browser. What do you check?

Easy answer: Check CORS origin, credentials configuration, preflight response, allowed methods/headers, HTTPS/mixed-content issues and the exact browser console/network error. Postman is not subject to browser CORS enforcement.

Code / example:

```
app.use(cors({origin:"https://app.example.com",credentials:true}));
```

353. A user edits another user's resource. How do you stop it?

Easy answer: Do not trust an ID sent from the frontend. Derive the authenticated user from the credential and query/update only records owned by that user or allowed by role.

Code / example:

```
await Job.updateOne({_id:req.params.id, employer:req.user.id},{ $set:update});
```

354. Two users submit the same data at the same time. How do you prevent duplicates?

Easy answer: Use database uniqueness constraints where appropriate and design the operation so race conditions cannot create duplicate records. Application-level "find then insert" alone is not enough.

Code / example:

```
db.users.createIndex({email:1},{unique:true});
```

355. Server memory keeps increasing. What do you investigate?

Easy answer: Look for unbounded caches, retained objects, event listeners, timers, large arrays, open resources and leaks in long-lived processes. Use profiling/heap snapshots rather than guessing.

Code / example:

```
setInterval(()=>cache.push(load()),1000); // suspicious if unbounded
```

356. An API is being brute-forced. What do you do?

Easy answer: Rate-limit the endpoint, add progressive delays/lockouts where appropriate, use strong password hashing, monitor failures, and consider IP/device/account signals. Do not reveal whether an email exists unnecessarily.

Code / example:

```
rateLimit({windowMs:15*60*1000,max:20});
```

357. MongoDB is unavailable. What should the API return?

Easy answer: Return a server-side availability error such as 503 when appropriate, log the incident, avoid exposing raw database errors, and use appropriate retry/health-check behavior.

Code / example:

```
res.status(503).json({message:"Service temporarily unavailable"});
```

358. An API is too slow. What is your approach?

Easy answer: Measure latency by endpoint, break it into network/auth/business/database time, inspect slow queries, reduce payloads and remove unnecessary work. Optimize based on evidence.

Code / example:

```
console.time("query");
await Job.find(filter);
console.timeEnd("query");
```

JavaScript Coding

359. Reverse a string.

Easy answer: A simple solution is split/reverse/join. In an interview, also explain time and space complexity.

Code / example:

```
function reverse(s){
  return s.split("").reverse().join("");
}
```

360. Check whether a string is a palindrome.

Easy answer: Compare characters from both ends and move inward. This uses $O(n)$ time and $O(1)$ extra space if indexed directly.

Code / example:

```
function isPalindrome(s){
  let l=0,r=s.length-1;
  while(l<r){ if(s[l]!==s[r]) return false; l++; r--; }
  return true;
}
```

361. Find the largest number in an array.

Easy answer: Track the largest value while scanning once. This is $O(n)$.

Code / example:

```
function max(arr){
  let best=-Infinity;
  for(const x of arr) best=Math.max(best,x);
  return best;
}
```

362. Find the second largest number.

Easy answer: Track the largest and second largest distinctly. Decide how you want duplicates handled before coding.

Code / example:

```
function secondLargest(arr){
  let first=-Infinity, second=-Infinity;
  for(const x of arr){
    if(x>first){ second=first; first=x; }
    else if(x>second && x<first){ second=x; }
  }
  return second;
}
```

363. Count frequency of elements.

Easy answer: Use a Map so each element can map to its count. This is $O(n)$ expected time.

Code / example:

```
function frequency(arr){
  const m=new Map();
  for(const x of arr) m.set(x,(m.get(x)||0)+1);
  return m;
}
```

364. Find duplicates in an array.

Easy answer: Track seen values in a Set; if a value is already present, it is a duplicate.

Code / example:

```
function duplicates(arr){
  const seen=new Set(), dup=new Set();
  for(const x of arr){ if(seen.has(x)) dup.add(x); else seen.add(x); }
  return [...dup];
}
```

365. Remove duplicates from an array.

Easy answer: For primitive values, Set is simplest. In interviews, also know the manual Set/Map approach and the two-pointer approach for sorted arrays.

Code / example:

```
const unique=[...new Set([1,2,2,3,3])];
```

366. Find the missing number from 0..n.

Easy answer: Use the expected sum $n(n+1)/2$ minus the actual sum, or XOR to avoid overflow concerns in some languages.

Code / example:

```
function missing(nums){
  const n=nums.length;
  let expected=n*(n+1)/2;
  let actual=nums.reduce((a,b)=>a+b,0);
  return expected-actual;
}
```

367. Two Sum.

Easy answer: Store previous numbers in a Map and look for target-current. This reduces a brute-force $O(n^2)$ search to expected $O(n)$.

Code / example:

```
function twoSum(nums,target){
  const map=new Map();
  for(let i=0;i<nums.length;i++){
    const need=target-nums[i];
    if(map.has(need)) return [map.get(need),i];
    map.set(nums[i],i);
  }
  return [];
}
```

368. Check if two strings are anagrams.

Easy answer: Count character frequencies or sort both strings. Frequency counting is usually $O(n)$ expected time.

Code / example:

```
function anagram(a,b){
  if(a.length!==b.length) return false;
  const m=new Map();
  for(const c of a) m.set(c,(m.get(c)||0)+1);
  for(const c of b){
    if(!m.has(c)) return false;
    m.set(c,m.get(c)-1);
    if(m.get(c)<0) return false;
  }
  return true;
}
```

369. Move all zeros to the end.

Easy answer: Use a pointer for the next non-zero position, then fill the rest with zeros. This can be done in $O(n)$ time.

Code / example:

```
function moveZeros(a){
  let p=0;
  for(const x of a) if(x!==0) a[p++]=x;
  while(p<a.length) a[p++]=0;
  return a;
}
```

370. Rotate an array.

Easy answer: A common $O(n)$ solution is the reverse trick: reverse all, reverse first k, reverse the remainder.

Code / example:

```
function rotate(a,k){
  k%=a.length;
  const rev=(l,r)=>{while(l<r){[a[l],a[r]]=a[r],a[l];l++;r--}};
  rev(0,a.length-1); rev(0,k-1); rev(k,a.length-1);
}
```

```

    return a;
  }

```

371. Flatten an array.

Easy answer: Use recursion for nested arbitrary depth or `Array.prototype.flat` when allowed. For interviews, understand stack/recursion and complexity.

Code / example:

```

function flatten(arr){
  const out=[];
  for(const x of arr) Array.isArray(x) ? out.push(...flatten(x)) : out.push(x);
  return out;
}

```

372. Find intersection of two arrays.

Easy answer: Use a Set for one array and check values from the other. If duplicates matter, use frequency counts instead.

Code / example:

```

function intersection(a,b){
  const s=new Set(a);
  return [...new Set(b.filter(x=>s.has(x)))];
}

```

373. Binary search in a sorted array.

Easy answer: Keep left/right bounds and compare the middle value to the target. Each step halves the search space, giving $O(\log n)$.

Code / example:

```

function binarySearch(a,target){
  let l=0,r=a.length-1;
  while(l<=r){
    const m=Math.floor((l+r)/2);
    if(a[m]===target) return m;
    if(a[m]<target) l=m+1; else r=m-1;
  }
  return -1;
}

```

374. Implement a debounce function.

Easy answer: Keep the timer ID in a closure. Each call clears the previous timer and starts a new one.

Code / example:

```

function debounce(fn,delay){
  let timer;
  return (...args)=>{
    clearTimeout(timer);
    timer=setTimeout(()=>fn(...args),delay);
  };
}

```

375. Implement a throttle function.

Easy answer: Track the last execution time and only allow the function to run when the interval has passed.

Code / example:

```

function throttle(fn,delay){
  let last=0;
  return (...args)=>{
    const now=Date.now();
    if(now-last>=delay){ last=now; fn(...args); }
  };
}

```

376. Implement Promise.all conceptually.

Easy answer: Store results by input index, count fulfilled tasks and reject once. In production, use the built-in `Promise.all` unless implementing it is explicitly requested.

Code / example:

```

function myAll(promises){
  return new Promise((resolve,reject)=>{
    const out=[], total=promises.length; let done=0;
    if(!total) return resolve([]);
    promises.forEach((p,i)=>Promise.resolve(p).then(v=>{
      out[i]=v; if(++done===total) resolve(out);
    },reject));
  });
}

```

Advanced JavaScript

377. What is execution context?

Easy answer: An execution context is the environment in which JavaScript code executes. It includes information such as variables, scope and this. Global and function execution contexts are created during execution.

Code / example:

```
function greet(name){
  const message=`Hi ${name}`;
  return message;
}
```

378. What is the lexical environment?

Easy answer: It is the structure used to resolve variable names for a piece of code, together with a reference to its outer environment. This is the foundation of lexical scope and closures.

Code / example:

```
const x=1;
function f(){ console.log(x); }
```

379. What is the prototype chain?

Easy answer: When a property is not found directly on an object, JavaScript looks at its prototype, then that prototype's prototype, continuing until null.

Code / example:

```
const arr=[];
arr.push(1); // found via Array.prototype
```

380. What is event delegation?

Easy answer: It is attaching a listener to a common ancestor rather than every child, relying on event bubbling. It reduces listeners and handles dynamic children.

Code / example:

```
parent.addEventListener("click",e=>{
  if(e.target.matches("button")) ...
});
```

381. What is a memory leak in JavaScript?

Easy answer: It is memory that is no longer logically needed but remains reachable, preventing garbage collection. Common web causes include forgotten listeners, timers, large caches and references held by global objects.

Code / example:

```
const cache=[];
setInterval(()=>cache.push(hugeObject),1000);
```

382. What is destructuring with defaults?

Easy answer: You can provide fallback values while extracting fields. This is useful when API responses may omit optional properties.

Code / example:

```
const {page=1, limit=20}=options;
```

383. What are optional chaining and nullish coalescing together?

Easy answer: Optional chaining safely accesses nested values, while nullish coalescing supplies a default only for null/undefined. Together they make defensive reads concise.

Code / example:

```
const city=user?.address?.city ?? "Unknown";
```

384. What is referential equality?

Easy answer: Objects/arrays are compared by reference, not by deep content. Two separately created objects with the same fields are not ===.

Code / example:

```
{ } === { } // false
const a={}; const b=a; a===b // true
```

385. What is immutability and why is it useful in React?

Easy answer: Immutability means not changing existing data in place. Creating new references makes state changes easier to detect and reason about, and it supports predictable updates.

Code / example:

```
setUser({...user, name: "New"});
```

386. What is a race condition in async JavaScript?

Easy answer: A race condition occurs when the final result depends on the order asynchronous operations finish. Use cancellation, request IDs or state checks when stale responses could overwrite newer data.

Code / example:

```
const requestId=++latestRequestId;
const data=await load();
if(requestId!==latestRequestId) return;
```

387. What is AbortController?

Easy answer: AbortController lets you cancel certain asynchronous operations such as fetch. It is useful when a component unmounts or a newer search supersedes an older request.

Code / example:

```
const controller=new AbortController();
fetch(url,{signal:controller.signal});
controller.abort();
```

388. What is a microtask starvation problem?

Easy answer: If code continuously schedules microtasks, the runtime can keep draining them and delay timers/rendering. Keep microtask chains bounded and avoid recursive Promise scheduling.

Code / example:

```
function loop(){ Promise.resolve().then(loop); }
// dangerous: can starve tasks
```

389. What is tail recursion?

Easy answer: Tail recursion places the recursive call as the final action. JavaScript engines do not universally optimize tail calls in the way some languages do, so do not assume it prevents stack overflow.

Code / example:

```
function fact(n,acc=1){ return n<=1 ? acc : fact(n-1,acc*n); }
```

390. What is currying vs partial application?

Easy answer: Currying transforms a multi-argument function into chained one-argument functions. Partial application pre-fills some arguments and returns a function that accepts the rest.

Code / example:

```
const multiply=(a,b)=>a*b;
const double=(b)=>multiply(2,b);
```

391. What is the difference between synchronous CPU work and async I/O in Node?

Easy answer: Async I/O can let the event loop handle other work while the OS or worker pool waits. Heavy CPU loops still block the main JavaScript thread unless moved to worker threads/processes.

Code / example:

```
for(let i=0;i<1e10;i++){ // blocks
  await fs.promises.readFile("file"); // async I/O
```

Advanced React

392. What is React Fiber?

Easy answer: Fiber is the internal reconciliation architecture React uses to represent work and schedule rendering. It enables React to split/prioritize work and supports newer rendering capabilities.

Code / example:

```
// You use the React API; Fiber is an internal implementation detail.
```

393. Does a parent re-render always mean the child DOM changes?

Easy answer: No. A child component may re-render, but React can determine that the browser DOM needs little or no change. Memoization can also skip some child renders.

Code / example:

```
const Child=React.memo(({value})=><span>{value}</span>);
```

394. What is state batching?

Easy answer: React may group multiple state updates so they produce fewer renders. Functional updates are important when several updates depend on the previous state.

Code / example:

```
setCount(c=>c+1);
setCount(c=>c+1);
```

395. What is hydration?

Easy answer: Hydration is attaching React behavior/event handlers to HTML that was already rendered on the server. It is common with SSR frameworks.

Code / example:

```
// Server HTML -> React hydrates it in the browser.
```

396. What is virtualization?

Easy answer: Virtualization renders only the rows currently visible in a huge list rather than all thousands of DOM nodes. It can greatly reduce memory and rendering work.

Code / example:

```
// Libraries such as react-window can virtualize large lists.
```

397. How do you prevent duplicate API calls?

Easy answer: Centralize data fetching, keep effect dependencies correct, use request deduplication/caching, cancel stale requests and avoid firing the same request from multiple mounted components unnecessarily.

Code / example:

```
const {data}=useGetJobsQuery(); // cached/deduped by RTK Query
```

398. How do you optimize large lists?

Easy answer: Use virtualization, pagination/infinite loading, stable keys, lightweight row components and memoization only where measurements show benefit.

Code / example:

```
items.map(item=><Row key={item.id} item={item}/>);
```

399. What is a stale closure in React?

Easy answer: A callback can capture values from the render in which it was created. This may lead to stale state in timers/effects. Use functional updates, correct dependencies or refs when appropriate.

Code / example:

```
setCount(c=>c+1); // avoids stale captured count
```

400. What is StrictMode?

Easy answer: React StrictMode enables additional development-time checks and may re-run certain lifecycle/effect patterns to expose unsafe side effects. It does not double-render production behavior in the same way.

Code / example:

```
<React.StrictMode><App /></React.StrictMode>
```

401. Why should keys be stable?

Easy answer: React uses keys to match an item across renders. A stable unique key preserves the correct component identity and local state when lists change.

Code / example:

```
items.map(x=><Row key={x.id} />);
```

402. What is a custom hook good for?

Easy answer: It extracts reusable stateful behavior such as fetching, online status, form logic or subscriptions while keeping the UI component focused on presentation.

Code / example:

```
function useDebounceValue(value, delay){ ... }
```

403. How do you handle global auth state?

Easy answer: You can use Context for simpler apps or Redux Toolkit/another state solution for larger client state. The server remains the authority for actual permission checks.

Code / example:

```
const auth=useContext(AuthContext);
```

404. How do you handle API caching?

Easy answer: Use RTK Query, React Query or a carefully designed cache layer to deduplicate requests, reuse data and invalidate stale results.

Code / example:

```
const {data}=useQuery({queryKey:["jobs"],queryFn:getJobs});
```

405. What is optimistic UI?

Easy answer: The UI updates immediately as if the server action succeeded, then rolls back if the server rejects it. It improves perceived speed but requires careful failure handling.

Code / example:

```
setLiked(true);
try{ await like(); } catch{ setLiked(false); }
```

406. What is error boundary?

Easy answer: Error boundaries catch certain rendering/lifecycle errors in a component subtree and show fallback UI. They do not replace try/catch for ordinary event-handler/API errors.

Code / example:

```
<ErrorBoundary fallback={<ErrorPage/>}>
  <Dashboard/>
</ErrorBoundary>
```

API Design

407. How would you design a jobs API?

Easy answer: Model jobs as resources with endpoints for listing, detail, creation, update and deletion. Add filtering/pagination via query parameters and enforce employer permissions on write endpoints.

Code / example:

```
GET /api/jobs
GET /api/jobs/:id
POST /api/jobs
PATCH /api/jobs/:id
DELETE /api/jobs/:id
```

408. How would you design authentication endpoints?

Easy answer: Keep a small, clear set such as register, login, logout, current-user/profile and refresh when using refresh tokens. Document auth behavior and status codes.

Code / example:

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
GET /api/auth/me
```

409. How would you design job applications?

Easy answer: Treat applications as a relationship/resource. The backend should prevent duplicate applications and ensure users can only create/view actions allowed by role/ownership.

Code / example:

```
POST /api/jobs/:jobId/applications
GET /api/applications
```

410. What makes an API response consistent?

Easy answer: Use consistent success/error shapes, status codes, pagination metadata, validation messages and request identifiers where useful. Consistency reduces frontend complexity.

Code / example:

```
{ "success": true, "data": jobs, "page": 1, "limit": 20 }
```

411. What is idempotency and why useful?

Easy answer: An operation is idempotent if repeating the same request has the same intended effect as doing it once. PUT is designed around idempotent semantics; idempotency keys can make retryable POST workflows safer.

Code / example:

```
Idempotency-Key: abc-123
```

412. How do you version an API?

Easy answer: Common approaches are URL versioning, headers or media types. Pick a clear strategy and document breaking changes.

Code / example:

```
/api/v1/jobs
/api/v2/jobs
```

413. How do you filter/sort/paginate?

Easy answer: Use query parameters with a predictable grammar and server-side validation. Do not accept raw MongoDB filter objects from clients.

Code / example:

```
GET /jobs?city=Delhi&sort=-createdAt&limit=20
```

414. How do you protect an API from mass assignment?

Easy answer: Whitelist allowed fields rather than blindly spreading request.body into database updates. Sensitive fields such as role/owner should be controlled by server logic.

Code / example:

```
const update={title,description};
await Job.updateOne({_id:id,employer:req.user.id},{set:update});
```

415. How do you design an upload endpoint?

Easy answer: Authenticate, validate file size/type, stream or upload directly to object storage when practical, store metadata, and enforce who may access the file.

Code / example:

```
POST /api/resumes
Content-Type: multipart/form-data
```

416. What logs are useful in production?

Easy answer: Log request IDs, route, status, latency, structured error details and important domain events without logging passwords or sensitive tokens. Centralized logs make debugging much easier.

Code / example:

```
logger.info({requestId,route,status,ms},"request");
```

HR / Behavioral

417. Tell me about yourself.

Easy answer: Give a 60–90 second summary: education/background, core technical skills, internship/project experience, strongest relevant work and what role you are looking for. Keep it professional and connected to the job.

Code / example:

```
"I am a BTech CSE student focused on MERN..."
```

418. Why should we hire you?

Easy answer: Connect your evidence to the role: hands-on MERN projects, JavaScript/React/Node knowledge, ability to debug and willingness to learn. Give one concrete example instead of only adjectives.

Code / example:

```
"I have built APIs, authentication and database-backed features, and I can explain how each part works."
```

419. Why MERN?

Easy answer: Say why it fits your experience: you can use JavaScript across the stack, React supports reusable UI, Node/Express are productive for APIs, and MongoDB fits document-oriented application data.

Code / example:

```
React -> Express/Node -> MongoDB
```

420. What are your strengths?

Easy answer: Choose 2–3 strengths relevant to software development, then prove each with a short example such as debugging, persistence, learning or communication.

Code / example:

```
Strength -> Example -> Result
```

421. What is your weakness?

Easy answer: Choose a genuine weakness that is not fatal to the role and show a concrete improvement method. Avoid fake answers like "I work too hard."

Code / example:

```
Weakness -> action -> measurable improvement
```

422. Where do you see yourself in five years?

Easy answer: Focus on becoming a strong engineer, owning features/systems, learning from experienced teammates and contributing to product outcomes. Avoid focusing only on title.

Code / example:

"I want to grow into an engineer who can own end-to-end features."

423. Why do you want this company?

Easy answer: Mention the product, engineering exposure, role responsibilities or learning opportunity. Show that you read the job description and are not giving a generic answer.

Code / example:

Company product -> role -> your skill match

424. Tell me about a difficult bug.

Easy answer: Use: situation, symptoms, investigation, root cause, fix, verification and lesson learned. Interviewers are testing debugging mindset, not whether you never make mistakes.

Code / example:

Symptom -> logs/network -> root cause -> fix -> test

425. Tell me about a failure.

Easy answer: Choose a real but manageable example. Own your part, explain what changed afterward and avoid blaming others.

Code / example:

What happened -> what I learned -> what I changed

426. How do you handle disagreement in a team?

Easy answer: Clarify the goal, compare evidence/trade-offs, listen, propose a small experiment when possible and support the final team decision.

Code / example:

Goal -> options -> evidence -> decision

427. What salary do you expect?

Easy answer: Give a researched range when possible and show flexibility based on role, responsibilities and total package. For fresher roles, emphasize fit and growth without underselling yourself.

Code / example:

"I am open to the company's range for this role; based on the market and responsibilities, I would expect around X-Y."

428. Are you willing to relocate?

Easy answer: Answer honestly. If you have flexibility, state the locations/conditions you can accept.

Code / example:

"Yes, I am open to relocation for the right role."

429. When can you join?

Easy answer: State your actual availability and any genuine commitments. Do not promise a joining date you cannot meet.

Code / example:

"I can join after my current commitments are completed."

430. Do you have any questions for us?

Easy answer: Ask about the first 90 days, team structure, code review, deployment practices, mentorship and how success is measured. Strong questions show professional curiosity.

Code / example:

"What would success look like in the first three months?"

431. What if you do not know an answer in an interview?

Easy answer: Do not bluff. Say what you know, state your assumption, and explain how you would verify it. Then attempt a reasoned answer if possible.

Code / example:

"I have not used X directly, but I understand Y and I would verify Z before implementing it."

Final Interview Checklist

- Be able to explain your complete MERN architecture on paper.
- Know JavaScript scope, hoisting, closure, this, promises, async/await and the event loop.
- Know React state/props/hooks, forms, routing, Redux Toolkit and API data fetching.
- Know Express middleware, routing, controllers, validation and centralized errors.
- Know MongoDB CRUD, indexes, aggregation, references/embedding and Mongoose.
- Know HTTP status codes, REST, JWT, cookies, CORS, XSS, CSRF and password hashing.
- Know every project on your resume deeply enough to explain one API and one database flow line-by-line.
- Practice JavaScript coding aloud and include time/space complexity when asked.
- For unknown questions, be honest, reason from fundamentals and explain how you would verify.

Total numbered questions: 431